Modern Frontend Development

Welcome to the Jungle



This Year's Agenda

DAY 1

Tooling for a Modern Front-End Build

Modern JavaScript

Async JavaScript: Callbacks, Promises, RxJS

DAY 2

TypeScript

Web Platform APIs

Concepts of Single Page Applications

Overview "Big 3" Frontend Frameworks

Styling Mechanisms

TypeScript



TypeScript

TypeScript is JavaScript with syntax for types.

TypeScript is a strongly typed programming language that builds on JavaScript, giving you better tooling at any scale

Previous Tag-Lines on the project page:

- JavaScript that scales!
- TypeScript is a language for application-scale JavaScript development.

http://www.typescriptlang.org/

TypeScript 7 is written in Go

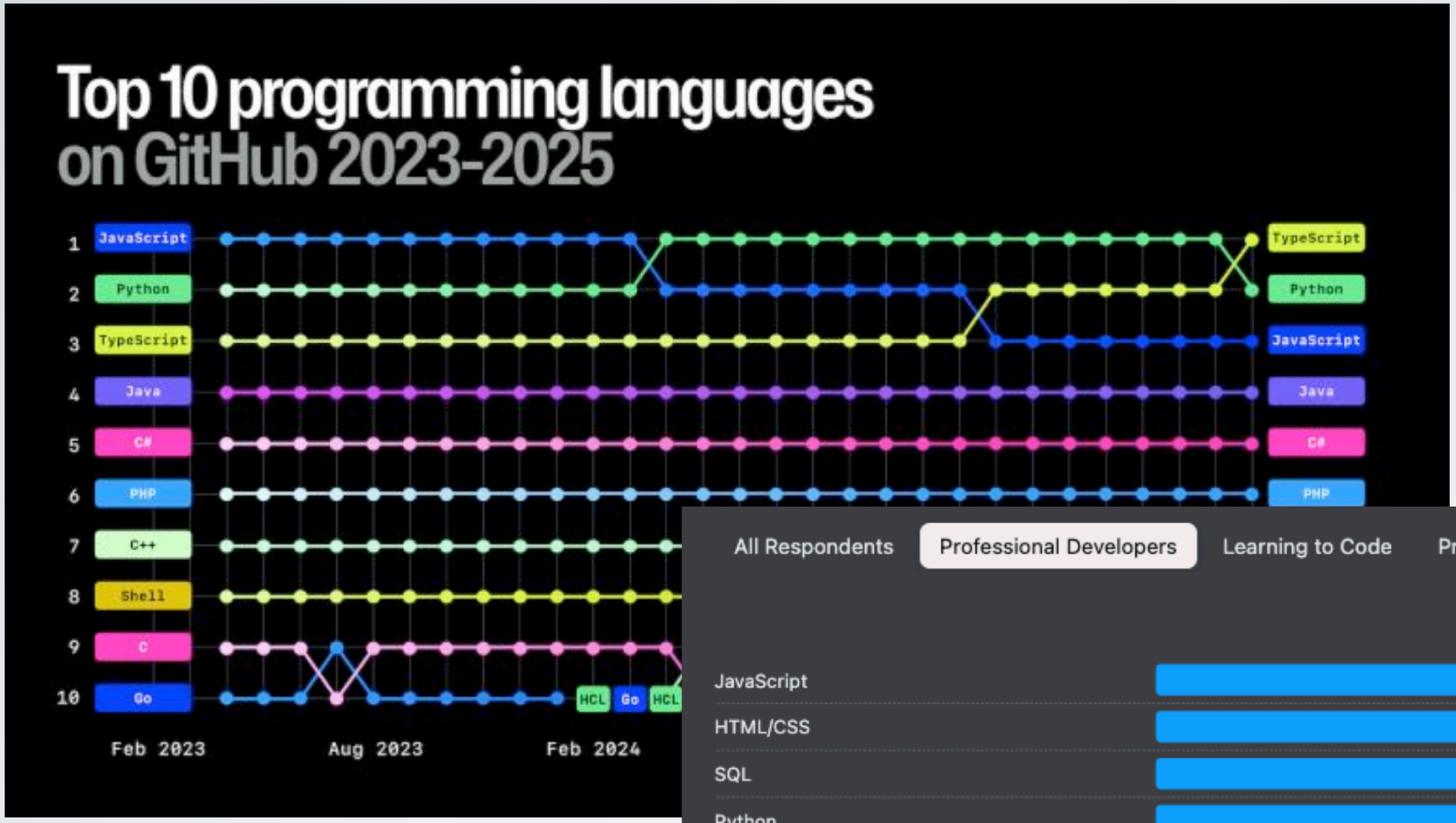
TypeScript 7.0 was released in July 2026
https://devblogs.microsoft.com/typescript/

- TypeScript compilation speedup between 8x and 12x
- The "IDE experience" becomes faster

Worth mentioning: TypeScript compiler in Rust
https://bun.com/blog/bun-v1.4.3
https://blog.cloudflare.com/typescript-rust-port/

Currently not officially supported by Angular.

The Rise of TypeScript





https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/
https://survey.stackoverflow.co/2025/technology#most-popular-technologies

ECMAScript Versions

- JavaScript: Initial Release in Netscape 2, 1995
- ECMAScript 1: 1997
- ECMAScript 2: 1998
- ECMAScript 3: 1999
- ~~ECMAScript 4~~: abandoned
- ECMAScript 5: 2009
- ECMAScript 2015: 2015
- ECMAScript 2016
- ECMAScript 2017
- …
- ECMAScript 2024
- ECMAScript 2025



https://262.ecma-international.org/

JavaScript was never engineered for large applications. It was intended for 100-1000 lines of codes.

Today we build apps with millions of lines of code. Very large JavaScript code bases tend to become “read-only”.

– Anders Hejlsberg, Build 2016

https://channel9.msdn.com/Events/Build/2016/B881



David K. 🎹
@DavidKPiano

I love TypeScript. It keeps me from coding too fast.

4:32 PM · Feb 19, 2021 · Twitter Web App

118 Retweets **23** Quote Tweets **1,615** Likes



David K. 🎹 @DavidKPiano · Feb 19
Replying to @DavidKPiano

JavaScript = move fast and break things
TypeScript = move slow and prevent things from breaking

15 33 280

https://twitter.com/DavidKPiano/status/1362787204058263558

TypeScript is "abstract"!

You can't "fix" a bug with TypeScript!

```
interface Person {
  firstName: string;
  movies: string[];
}

async function fetchLuke(): Promise<Person> {
  const lukeResponse = await fetch(`https://swapi.info/api/people/1`);
  const luke = await lukeResponse.json() as Person;
  return luke
}

async function main() {
  console.log("Starting ...");
  const luke = await fetchLuke();
  // 💥 KABOOM! - the following line throws a TypeError at runtime
  console.log(`Luke appears in ${luke.movies.length} movies.`);
}

main();
```

But correctly used TypeScript does prevent bugs ...



west, donavon west
@donavon

If you lie to TypeScript, TypeScript will lie to you.

7:23 PM · Aug 24, 2021 · Twitter Web App

https://twitter.com/donavon/status/1430219301500358658

```
interface Person {
  firstName: string;
  lastName: string;
}
```

using axios with TypeScript

```
const data = await axios.get<Person>(BACKEND_URL);
```

using ky with TypeScript

```
const data = await ky.get(BACKEND_URL).json<Person>();
```

using Angular HttpClient

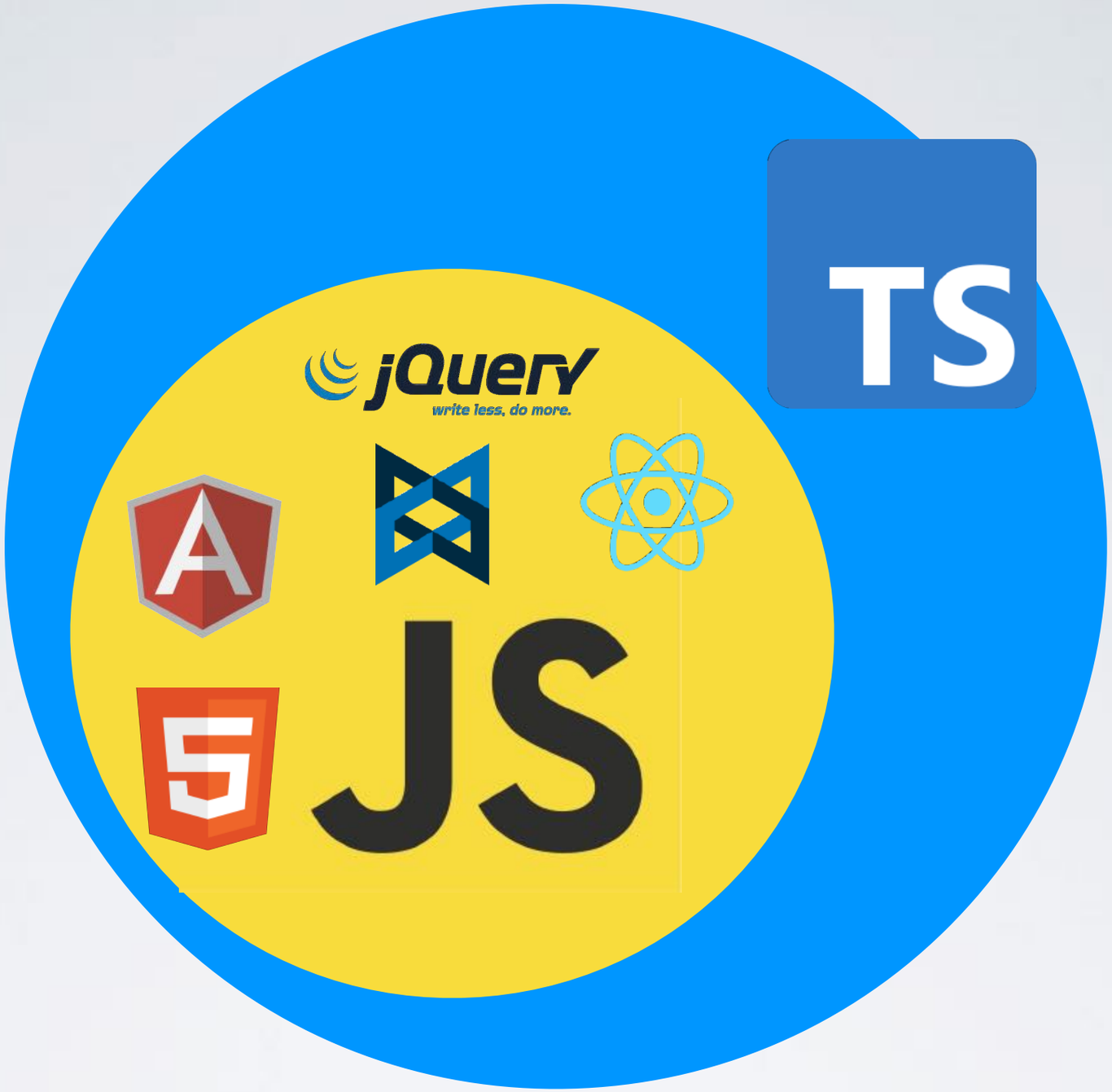
```
fetchData(): Observable<Person> {
  return this.http.get<Person>(BACKEND_URL);
}
```

The type information is only for the TypeScript compiler at build time. At runtime there is no guarantee or check that the network call really returns objects of the specified type.

You should only use **`interface`** or **`type`** as *type arguments* for backend access (not **`class`**).

The Goal of TypeScript in 2012





"TypeScript is a superset of JavaScript."

Any valid JavaScript (theoretically) is valid TypeScript. But TypeScript adds syntax for types, which is not valid JavaScript.

"TypeScript is a subset of JavaScript."

TypeScript throws buid-time errors for valid JavaScript, constraining the "flexibility" of JavaScript.

This is a differentiating factor compared other languages that compile to JavaScript: https://github.com/jashkenas/coffeescript/wiki/List-of-languages-that-compile-to-JS

TS is a superset of JS. Start with your existing JS and "enhance" it ...

It starts with JavaScript.

It ends with JavaScript.

After transpilation it's pure JS again. At runtime there is nothing left of TS!

TypeScript "Compilation"

At runtime TypeScript is just JavaScript.
Type information is stripped before runtime, typically at build-time.

TypeScript becomes JavaScript via the delete key.

```ts
type Result = "pass" | "fail"

function verify(result: Result) {
  if (result === "pass") {
    console.log("Passed")
  } else {
    console.log("Failed")
  }
}
```

TypeScript file.

```ts
type Result = "pass" | "fail"

function verify(result: Result) {
  if (result === "pass") {
    console.log("Passed")
  } else {
    console.log("Failed")
  }
}
```

Types are removed.

```ts
function verify(result) {
  if (result === "pass") {
    console.log("Passed")
  } else {
    console.log("Failed")
  }
}
```

JavaScript file.

https://www.typescriptlang.org/

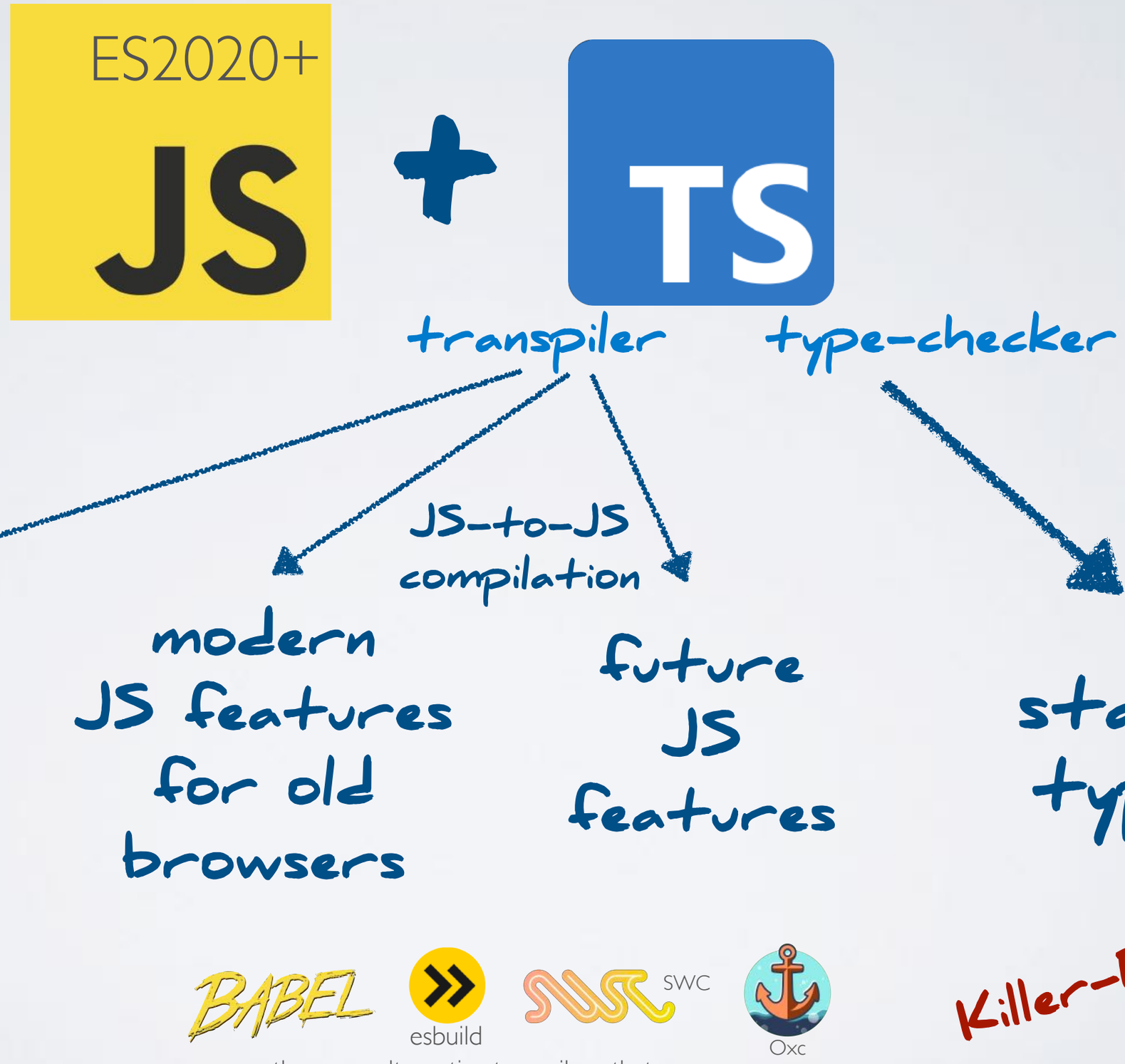
TypeScript Today

- … is a compiler that provides modern (ES2015+) language features for older Browsers
- … is a compiler that provides some future ESnext language features (i.e. decorators, `using` declarations …)
- … is a compiler that adds an optional static type system on top of JavaScript.
- … is also compiler that provides a few constructs that are not available in JavaScript (enums, namespaces) - however today it is "good practice" not to use these.
  Since TypesScript 5.8 this can be switched of with `erasableSyntaxOnly: true`



How does the TypeScript team try to avoid negative effects on the JS ecosystem? - https://www.youtube.com/watch?v=8qm49TyMUPI

TypeScript Today



see: `erasableSyntaxOnly`

there are alternative transpilers, that are used in many "state-of-the-art" toolchains

TypeScript Design Goals

https://github.com/Microsoft/TypeScript/wiki/TypeScript-Design-Goals

Impose no runtime overhead on emitted programs.

Align with current and future ECMAScript proposals.

Preserve runtime behavior of all JavaScript code.

Avoid adding expression-level syntax.

Use a consistent, fully erasable, structural type system.

Consequences:

- integrating existing JavaScript is easy.
- migrating from JavaScript is easy.
- *You can't be proficient in TypeScript without (deep) knowlege of JavaScript!*

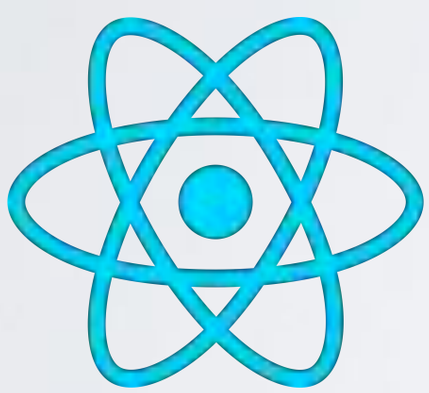
TypeScript sees itself as the Switzerland for JavaScript Frameworks

- Anders Hejlsberg, TypeScript: Static types for JavaScript
https://www.youtube.com/watch?v=ET4kT88JRXs



Angular is written in TypeScript and the whole Angular ecosystem and its community has fully embraced TypeScript.

But Angular versions have a tighter coupling to TypeScript versions than other frameworks:
https://angular.dev/reference/versions#actively-supported-versions



TypeScript support is especially strong for React, since React does not have it's own templating "language". JSX is directly supported by TypeScript in a typesafe way.



Vue 3 was a full rewrite in TypeScript
One of the goals was to provide better support for TypeScript projects.

TypeScript

Project Setup

Global installation:

~~`npm install -g typescript`~~ avoid!

Local installation:

```
npm init -y
npm install -D typescript
```

Configuration:

`npx tsc --init`

edit `tsconfig.json`

Usage:

run via: `npx tsc`

or via npm script: `tsc`

(modern frontend templates are setting up TypeScript, so typically you don't have to do this manually today)

tsconfig setup

The TypeScript compiler is configured in `tsconfig.json`

```
{
  "compilerOptions": {
   "strict": true,
    "noImplicitOverride": true,
    "noPropertyAccessFromIndexSignature": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "skipLibCheck": true,
    "isolatedModules": true,
    "experimentalDecorators": true,
    "importHelpers": true,
    "target": "ES2022",
    "module": "preserve"
}
```

(example generated by Angular CLI v20)

note: TypeScript v6 deprecates many configs!

Special mention: `erasableSyntaxOnly`

https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-8.html#the---erasablesyntaxonly-option

https://www.typescriptlang.org/tsconfig
https://www.typescriptlang.org/docs/handbook/tsconfig-json.html

Using Basic Types

Types are an optional, opt-in feature in TypeScript.

Basic Types:

```
let b:boolean = true;
let n:number = 42;
let s:string = 'Hello!';
let a1: number[] = [1,2,3];
let a2: Array<string> = ['Hello','World', '!'];
enum ArrowDirection { Up, Down, Left, Right}
let e: ArrowDirection = ArrowDirection.Up;
```

Basic type checking:

```
let x:string = 'Hello World';
x = 42; // => compiler error
```

Type inference:

```
let x = 'Hello World'; // the type of x is 'string'
x = 42; // => compiler error
```

The compiler will infer types if possible. It's considered a 'good style' to omit type annotations if they can be derived.

Built-In Types

TypeScript has built-in support for JavaScript types:

```
[1,2,3,4].reduce(
    (acc : {sum: number} , e : number ) => {
        return {sum: acc.sum + e}
    }, {sum: 0});
```

TypeScript has built-in support for browser APIs:

```
document.addEventListener( type: 'keydown',  listener: function(event : KeyboardEvent ) {
  console.log(event.clientX, event.clientY);
});
```

```
document.querySelector<HTMLDivElement>('#app')!.innerHtml = '';
```

TS2551: Property 'innerHtml' does not exist on type 'HTMLDivElement'. Did you mean 'innerHTML'?

lib.dom.d.ts(9130, 5): 'innerHTML' is declared here.

Rename reference ⌥⇧↩ More actions... ⌥↩

Installing 3rd-Party Type Definitions

Some libraries are not written in TypeScript.
Their type definitions can still be available via npm:

```
npm install @types/jquery
```

(`@types` is called a "scope" in npm)

The type definitions are maintained in the Definitely Typed repository:

- https://github.com/DefinitelyTyped/DefinitelyTyped/tree/master/types
- https://definitelytyped.org/

Advanced Types

Union Types:

```
let pet: IBird | IFish = {fly(){}, layEggs(){}};
pet = new Fish();
```

Intersection Types:

```
let flyingFish: IFish & IBird = {swim(){}, fly(){}, layEggs(){}};
flyingFish.swim();
```

Type Guards:

```
(pet as IFish).swim();
(<IFish>pet).swim();

if (pet instanceof Fish){
    pet.swim();
}
```

User defined type-guards:

```
function isFish(pet: Fish | Bird): pet is Fish {
    return (pet as any).swim !== undefined;
}
```

```
if (isFish(pet)) {
    pet.swim();
} else {
    pet.fly();
}
```

any Type

Types are optional. TypeScript allows you to to gradually opt-in and opt-out of type-checking during compilation.

```
let v: any = 42;
v = 'Hello!'; // anything can be assigned to v

let n:number = 43;
n = v; // v can be assigned to anything!!!
```

Variables have the implicit type **`any`** if TypeScript does not know it's type.

Typing can be enforced by setting **`noImplicitAny: true`** in **`tsconfig.json`**

https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#any
https://www.typescriptlang.org/tsconfig#noImplicitAny

`never` & `unknown` Types

Some functions *never* return a value:

```
function fail(msg: string): never {
  throw new Error(msg);
}
```

The *unknown* type represents any value. But it is not legal to do anything with an *unknown* value:

```
function f1(a: any) {
  a.b(); // OK
}
function f2(a: unknown) {
  a.b(); //ERROR: Object is of type 'unknown'.
}
```

https://www.typescriptlang.org/docs/handbook/2/functions.html#never
https://www.typescriptlang.org/docs/handbook/2/functions.html#unknown

Type Narrowing

```
function padLeft(padding: number | string, input: string): string {
  if (typeof padding === "number") {
    return " ".repeat(padding) + input; // -> padding is of type "number"
  }
  return padding + input; // -> padding is of type "string"
}
```

There are many mechanisms how TypeScript performs type narrowing:

https://www.typescriptlang.org/docs/handbook/2/narrowing.html

```
function isFish(pet: Fish | Bird): pet is Fish {
  return (pet as Fish).swim !== undefined;
}
```

https://www.typescriptlang.org/docs/handbook/2/narrowing.html#using-type-predicates

Function Types

A function has a type defined by its signature

```
let operation: string = 'add';

let mathFunc: (x: number, y:number) => number;

function add(x: number, y:number) : number {
  return x + y;
}

const subtract = (x: number, y:number):number => x - y;

if (operation === 'add')
  mathFunc = add;
else
  mathFunc = subtract;

console.log(mathFunc(4,2))
```

See also: https://kentcdodds.com/blog/typescript-function-syntaxes

Note: functions with fewer parameters are assignable to functions that take more parameters.

https://github.com/Microsoft/TypeScript/wiki/FAQ#why-are-functions-with-fewer-parameters-assignable-to-functions-that-take-more-parameters

Function Overloads

In my experiance a feature that is rarely used / useful.

overload signatures

```
// declaration
function doIt(): number;
function doIt(arg1: number): number;
function doIt(arg1: number, arg2: string): number;
function doIt(arg1?: number, arg2?: string): number {
    return 42;
}

// usage options
doIt();
doIt(44);
doIt(44, 'test');
```

implementation signature

The implemantation signature must be compatible to all overlaod signatures.

Classes & Interfaces

With classes & interfaces we can create custom types in TypeScript

```
interface IPerson {
    name: string,
}

class Person implements IPerson {
    name: string = 'Tyler';
    private age: number = 42;

    getInfo() {
        console.log(this.name); // inspect `this`
        return this.name + this.age;
    }
}

const p1: IPerson = new Person();
const p2: Person = new Person();
```

Interfaces are a pure build-time construct. They are not present at runtime!

TypeScript Classes

Differences to JavaScript classes:

parameter property
(properties declared in constructor signature
Note: not supportet with `erasableSyntaxOnly`)

```
class Car {
  private distanceDriven: number = 0;

  constructor(private engine: Engine){}

  drive(): number {
    this.engine.start();
    this.increaseDistance(1);
    return this.distance;
  }

  private increaseDistance(amount: number): void {
    this.distanceDriven + amount;
  }
}
```

access modifiers
(private, public, protected)
(optional, default is public)

type anotations
(optional)

The Structural Type System

The structure of an objects defines the compatiblity with a type, the object does not have to be explicitly declaread as a type (aka. "Duck Typing")

```
interface IPerson {
    name: string,
    age: number
}

const p3: IPerson = {
    name: 'Tyler',
    age: 42
};
```

Type Aliases

https://www.typescriptlang.org/docs/handbook/advanced-types.html#type-aliases

Type Aliases give a name to a type.

Describe the shape of an object:

```
type Person = {
 firstName: string;
  lastName: string;
  greet: (message: string) => string
}
```

Name a union type:

```
type ID = number | string;
```

Enums in TypeScript

In many cases Enums are not recommended!
Use union types instead!

```ts
enum Status {
  Admin,
  User,
  Moderator,
}
```

```ts
type Status = "Admin" | "User" | "Moderator";
```

prefer!

```ts
enum Status {
  Admin = "Admin",
  User = "User",
  Moderator = "Moderator",
}
```

avoid!

emitted JavaScript

```ts
var Status;
(function (Status) {
    Status["Admin"] = "Admin";
    Status["User"] = "User";
    Status["Moderator"] = "Moderator";
})(Status || (Status = {}));
```

Note:
`enums` can't be used native Node / Bun / Deno and with `erasableSyntaxOnly: true`

Reason: https://fettblog.eu/tidy-typescript-avoid-enums/
More Info: https://stackoverflow.com/a/60041791/32749

In modern TypeScript, you may not need an enum when an object with `as const` could suffice:

https://www.typescriptlang.org/docs/handbook/enums.html#objects-vs-enums

```
const ODirection = {
  Up: 0,
  Down: 1,
  Left: 2,
  Right: 3,
} as const;
type Direction = typeof ODirection[keyof typeof ODirection];
```

```
function run(dir: Direction) {}

run(ODirection.Right);
```

Types Aliases vs. Interfaces

In most cases type alias is interchangeable with an interface.

equivalent

```ts
type Person = {
  firstName: string;
  lastName: string;
  greet: (message: string) => string
}
```

```ts
interface Person {
  firstName: string;
  lastName: string;
  greet: (message: string) => string
}
```

Special cases, not possible with interfaces:

```ts
type Second = number;
```

alias for primitive type

```ts
type Status = "Admin" | "User" | "Moderator";
```

union types

A simple strategy to decide what to use:
use interfaces to describe objects, use types for everything else.

Main difference: Interfaces can be defined/extended in multiple files. Types can't.
Differences: https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#differences-between-type-aliases-and-interfaces

TypeScript recommends to use interfaces over type aliases:
Better error messages: https://www.typescriptlang.org/play?q=378#example/types-vs-interfaces
Performance: https://www.typescriptlang.org/play?q=378#example/types-vs-interfaces

Marking values as readonly with `as const`

```
const obj = {
  direction: "up" as const,
  foo: {
    bar: 42,
  },
} as const;

obj.foo.bar = 43;
console.log(obj.direction);
```

compile error

type is "up"
not string!

```
const arr = [ 42, "test" ] as const;
arr.push(43);

function fn(num: number){
    console.log(num);
}

fn(arr[0]);
fn(arr[1]);
```

compile error

ok, because type is number

compile error because type is "test"/string

https://www.totaltypescript.com/concepts/as-const

https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes-func.html#readonly-and-const

Creating Types from Types

"Metaprogramming with Types"

https://www.typescriptlang.org/docs/handbook/2/types-from-types.html

Example:

```
const person = {
    firstName: 'Jonas',
    age:42,
    address: {street: 42}
};

type T1 = typeof person;
type T2 = keyof typeof person;
type T4 = T1['address']['street'];

function getVal(){
    return new Promise<T1>(resolve => {
        resolve(person)
    });
}

type T3 = keyof Awaited<ReturnType<typeof getVal>>;
```

https://www.zhenghao.io/posts/type-programming
https://medium.com/@imkelen009/advanced-usage-of-typescript-%E4%B8%80-5b102a459b8d
https://obaranovskyi.medium.com/10-typescript-features-you-might-not-be-using-yet-or-didnt-understand-d1f28888ea45

Crazy: TypeScript Types can run DOOM!

(game from 1993)

https://www.youtube.com/watch?v=0mCsluv5FXA

To prove that TypeScript Types ar Turing Complete, someone implemented a TypeScript program that is running DOOM:

calling `tsc` returns a TypeScript object that
represents the ASCII-Art of a frame of DOOM.

Rendering the first frame of DOOM took running the program for 12 days with 20 million type instatiations per second ...

https://github.com/MichiganTypeScript/typescript-types-only-wasm-runtime

TypeScript Decorators

Decorators are an upcoming ECMAScript feature that allow us to customize classes and their members in a reusable way.

Decorators are heavily used in Angular.

```
function loggedMethod(originalMethod: any,
                      context: ClassMethodDecoratorContext) {
    const methodName = String(context.name);

    function replacementMethod(this: any, ...args: any[]) {
        console.log(`LOG: Entering method '${methodName}'.`)
        const result = originalMethod.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`)
        return result;
    }

    return replacementMethod;
}
```

```
class Person {
    name: string;
    constructor(name: string) {
        this.name = name;
    }

    @loggedMethod
    greet() {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const p = new Person("Ron");
p.greet();
```

```
// Output:
//
//   LOG: Entering method.
//   Hello, my name is Ron.
//   LOG: Exiting method.
```

https://devblogs.microsoft.com/typescript/announcing-typescript-5-0/#decorators



Runtime Type-Safety

(typesafe backend access and input validation)

Build- vs Runtime Type Safety

```
interface Person {
  firstName: string;
  movies: string[];
}

async function fetchLuke(): Promise<Person> {
  const lukeResponse = await fetch(`https://swapi.info/api/people/1`);
  const p = z.parse(lukeResponse);
  const luke = (await lukeResponse.json()) as Person;
  return luke;
}

async function main() {
  console.log("Starting ...");
  const luke = await fetchLuke();
  // 💥 KABOOM! - the following line throws a TypeError at runtime
  console.log(`Luke appears in ${luke.movies.length} movies.`);
}

main();
```



type cast is not type-safe!



west, donavon west
@donavon

If you lie to TypeScript, TypeScript will lie to you.

7:23 PM · Aug 24, 2021 · Twitter Web App

https://twitter.com/donavon/status/1430219301500358658

```
interface Person {
  firstName: string;
  lastName: string;
}
```

using axios with TypeScript

```
const data = await axios.get<Person>(BACKEND_URL);
```

using ky with TypeScript

```
const data = await ky.get(BACKEND_URL).json<Person>();
```

using Angular HttpClient

```
fetchData(): Observable<Person> {
  return this.http.get<Person>(BACKEND_URL);
}
```

The type information is only for the TypeScript compiler at build time. At runtime there is no guarantee or check that the network call really returns objects of the specified type.

You should only use **`interface`** or **`type`** as *type arguments* for backend access (not **`class`**).

Options

Use TypeScript in the Backend

Generate TypeScript definitions based on the backend model

Use runtime type-checking

TypeScript Generators

Open API Generator: https://openapi-generator.tech/

https://github.com/OpenAPITools/openapi-generator

```
npx @openapitools/openapi-generator-cli \
    generate -i https://petstore.swagger.io/v2/swagger.json -g typescript-angular -o .
```

Generated code is very different for: `typescript-angular`, `typescript-fetch`, `typescript-axios` ...

Alternative: https://swagger.io/tools/swagger-codegen/ resp. https://github.com/swagger-api/swagger-codegen

Nx Plugin for Open API Generator: https://github.com/trumbitta/nx-trumbitta/tree/main/packages/nx-plugin-openapi

Orval - Restful Client Generator https://orval.dev/overview

Alternative simple generator for Java: https://github.com/vojtechhabarta/typescript-generator

not maintained?

```
public class Person {
    public String name;
    public int age;
    public boolean hasChildren;
    public List<String> tags;
    public Map<String, String> emails;
}
```



```
interface Person {
    name: string;
    age: number;
    hasChildren: boolean;
    tags: string[];
    emails: { [index: string]: string };
}
```

Very easy to configure in a Maven or Gradle build.
Can detect DTOs automatically for JAX-RS or DTOs can be specified via pattern.
Somehow configurable (mapping, naming ...)
Limitation: One single generated file / module containing all the types.

Graphql:
- https://graphql-code-generator.com/

.NET:
- NSwag: https://github.com/RicoSuter/NSwag, Swashbuckle: https://github.com/domaindrivendev/Swashbuckle.AspNetCore
- TypeWriter Visual Studio Extension: https://github.com/AdaskoTheBeAsT/Typewriter/
- "Handmade": https://github.com/lmcarreiro/cs2ts-example

Runtime Type-Checking

Runtime type-checks can only happen via JavaScript.

Modern validation libraries let you describe types/schemas for runtime checking in a way that lets TypeScript derive types at build time.

Zod: describe types (schemas) in TS code: https://zod.dev/
→ infer TS types from schemas

ArkType: describe types (schemas) in TS code: https://arktype.io/
→ better runtime performance and better and more concise and readable syntax than Zod

Valibot: describe types (schemas) in TS code: https://github.com/fabian-hiller/valibot
→ infer TS types from schemas / a smaller and modular alternative to Zod

Alternatives: io-ts, typia, joi, ajv, yup ...

The traditional Problem with Runtime Type-Checking

Using a user defined type guard with a type predicate:

https://www.typescriptlang.org/docs/handbook/2/narrowing.html#using-type-predicates

```
interface Person {
  name: string;
  films: string[];
}

function checkPersonSchema(obj: any): obj is Person {
  return !(obj.name === undefined || typeof obj.name !== 'string'
    || obj.films === undefined
    || typeof obj.films !== 'object' || !(obj.films instanceof Array)
    || obj.films.forEach((film: any) => typeof film !== 'string'));
}

async function fetchLuke() {
  const lukeResponse = await fetch(`https://swapi.info/api/people/1`);
  const luke = await lukeResponse.json();
  if(checkPersonSchema(luke)){
    return luke;
  }
  throw new Error('Invalid Person Object!');
}
function printMessage(person: Person) {
  console.log(`Luke appears in ${person.films.length} movies.` );
}

export async function main() {
  const luke = await fetchLuke();
  printMessage(luke);
}
```

compile time type definition → duplicated definition!

runtime type check

runtine check

compile time check



Zod Demo

"runtime type-checking"

```
import z from 'zod';

const personSchema = z.object({
  name: z.string(),
  films: z.array(z.string())
})
type Person = z.infer<typeof personSchema>;

async function fetchLuke() {
  const lukeResponse = await fetch(`https://swapi.info/api/people/1`);
  const luke = await lukeResponse.json();
  return personSchema.parse(luke);
}

function printMessage(person: Person) {
  console.log(`Luke appears in ${person.films.length} movies.` );
}

async function main() {
  const luke = await fetchLuke();
  printMessage(luke)
}
```

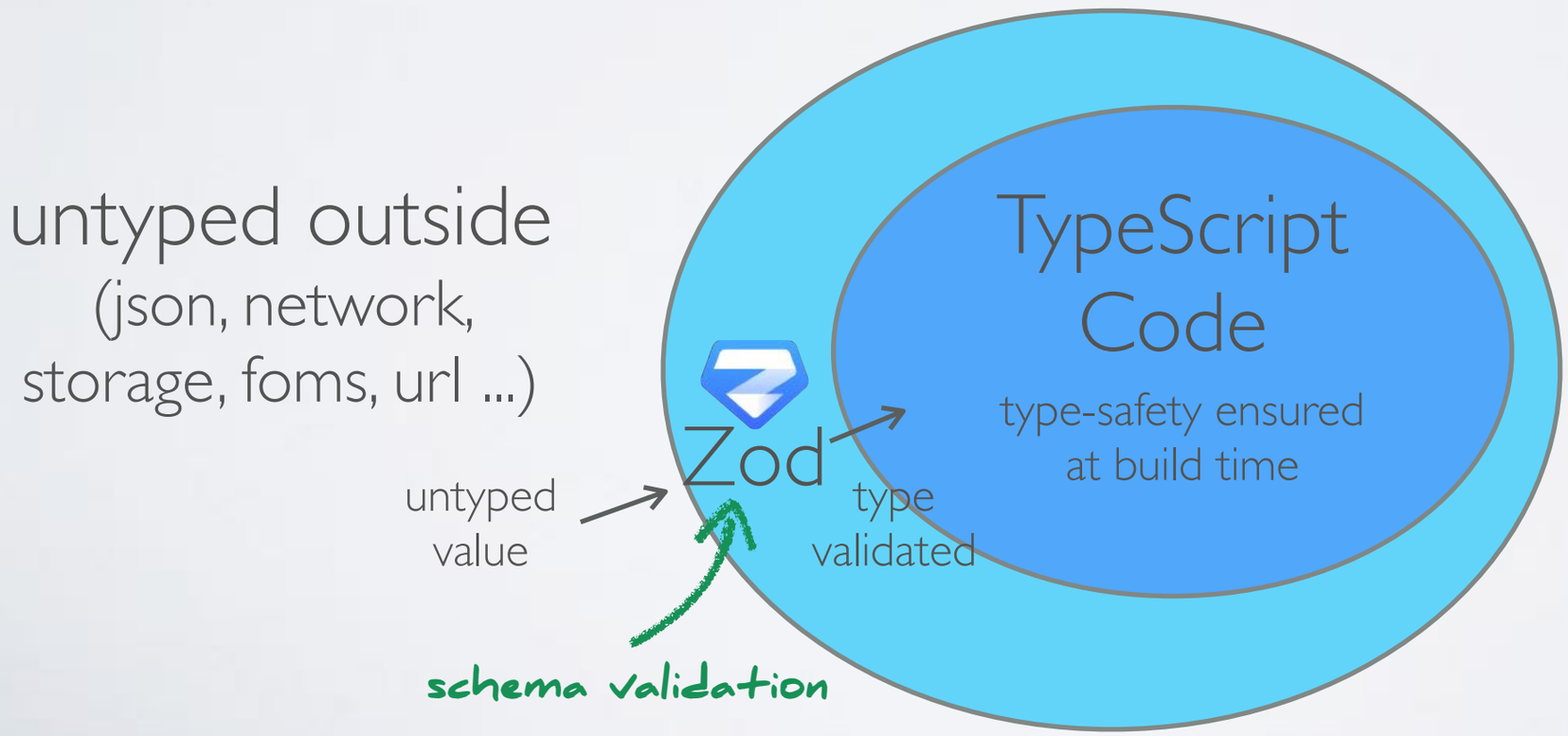
central definition!

runtine check

compile time check



Zod will detect the lie before it can cause any damage!





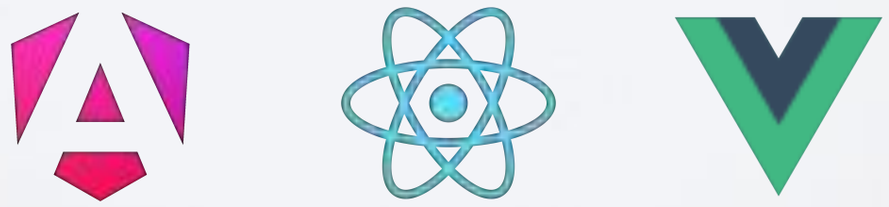
Web Platform APIs

Übungen

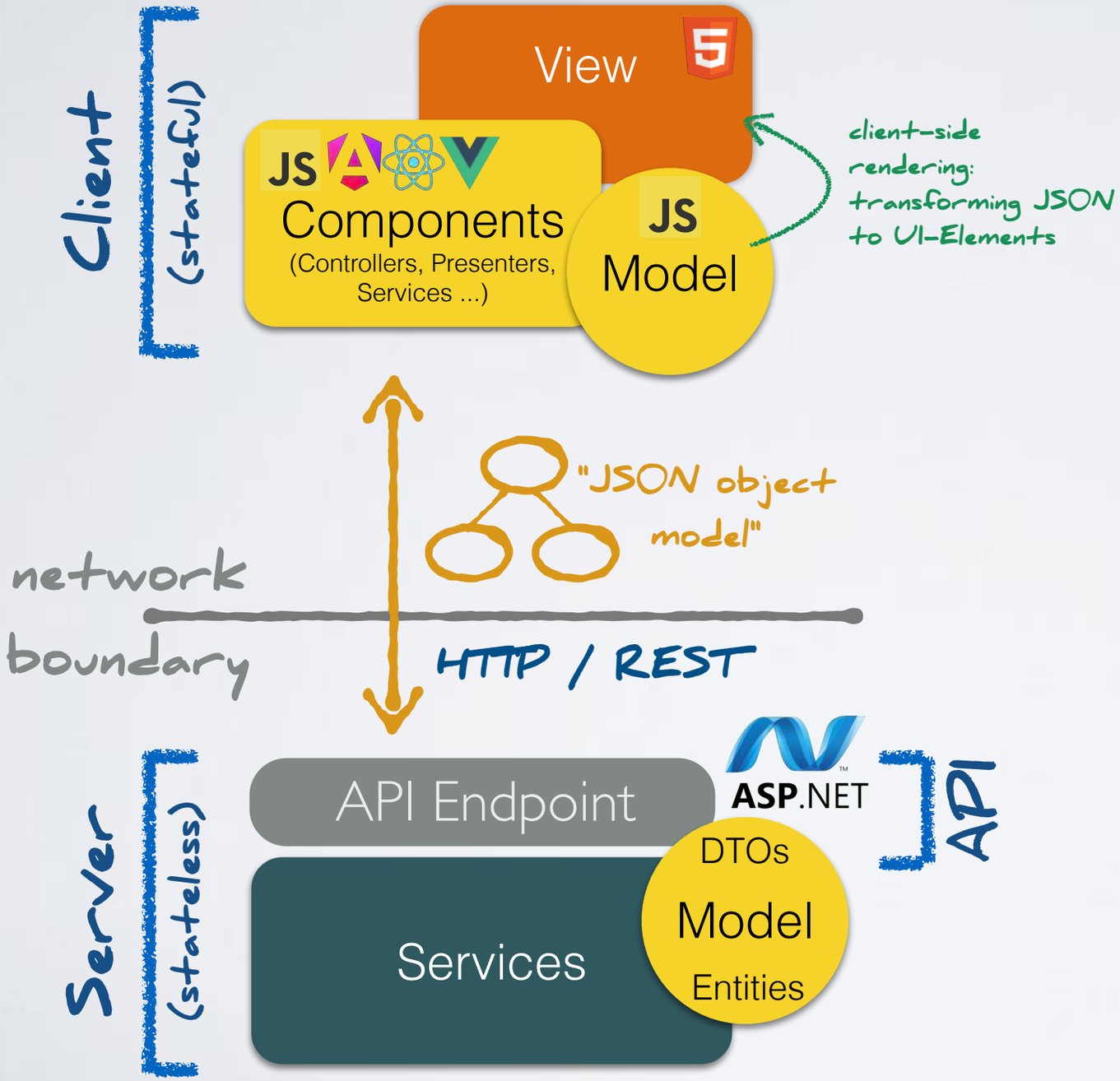
- fetch inkl SOP/CORS
- websockets
- streaming
- json
- forms
- web workers


NOT SURE IF APP IS CHANGING PAGES
WITHOUT ME NOTICING
OR APP IS A SPA
memegenerator.net

SPA Basics



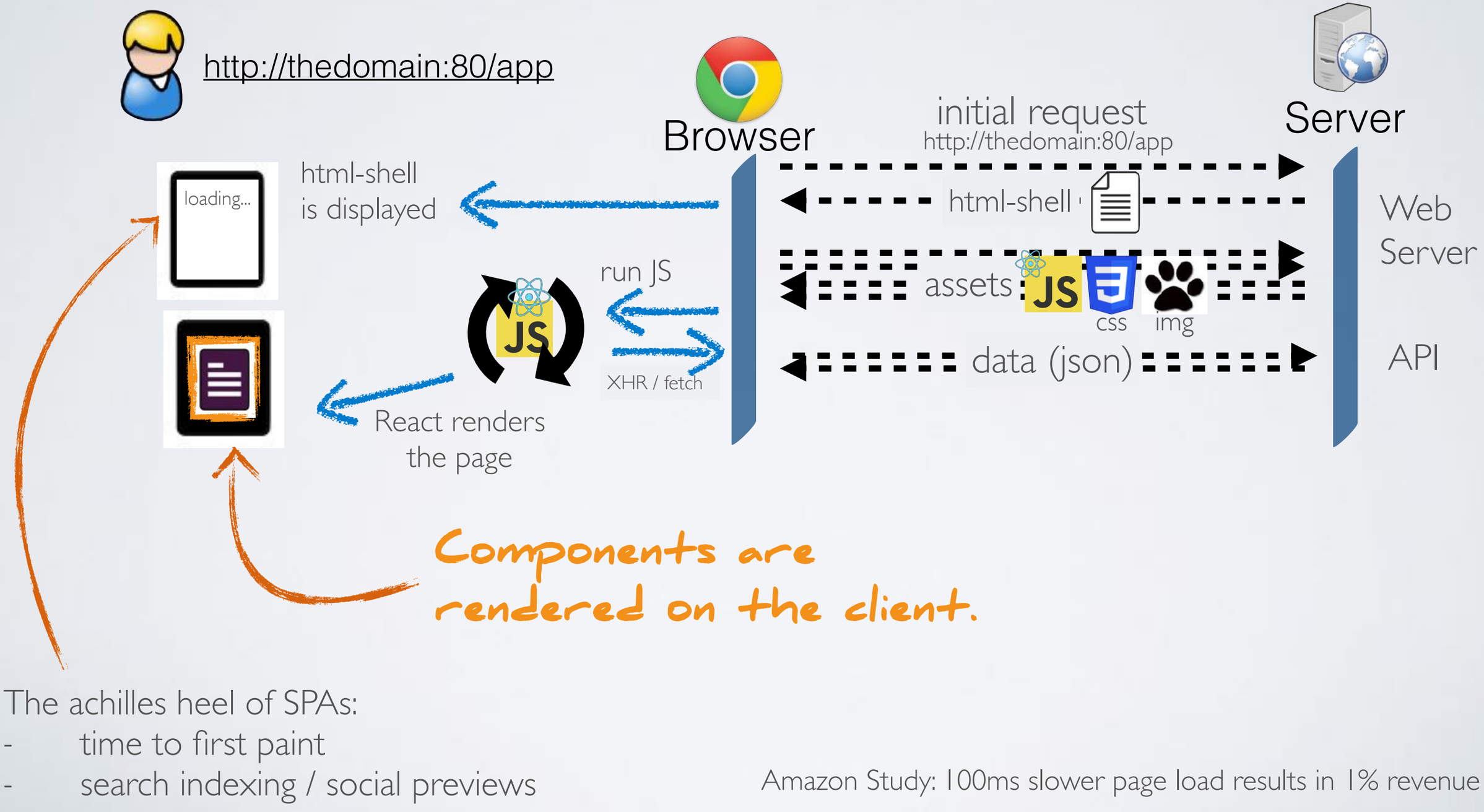
SPA Architecture



Rich client programming model in the browser.

Clear separation of concerns between client and server.

Traditional SPA:
Client Side Rendering



The achilles heel of SPAs:

- time to first paint
- search indexing / social previews

Amazon Study: 100ms slower page load results in 1% revenue loss!

SPA Data-Fetching

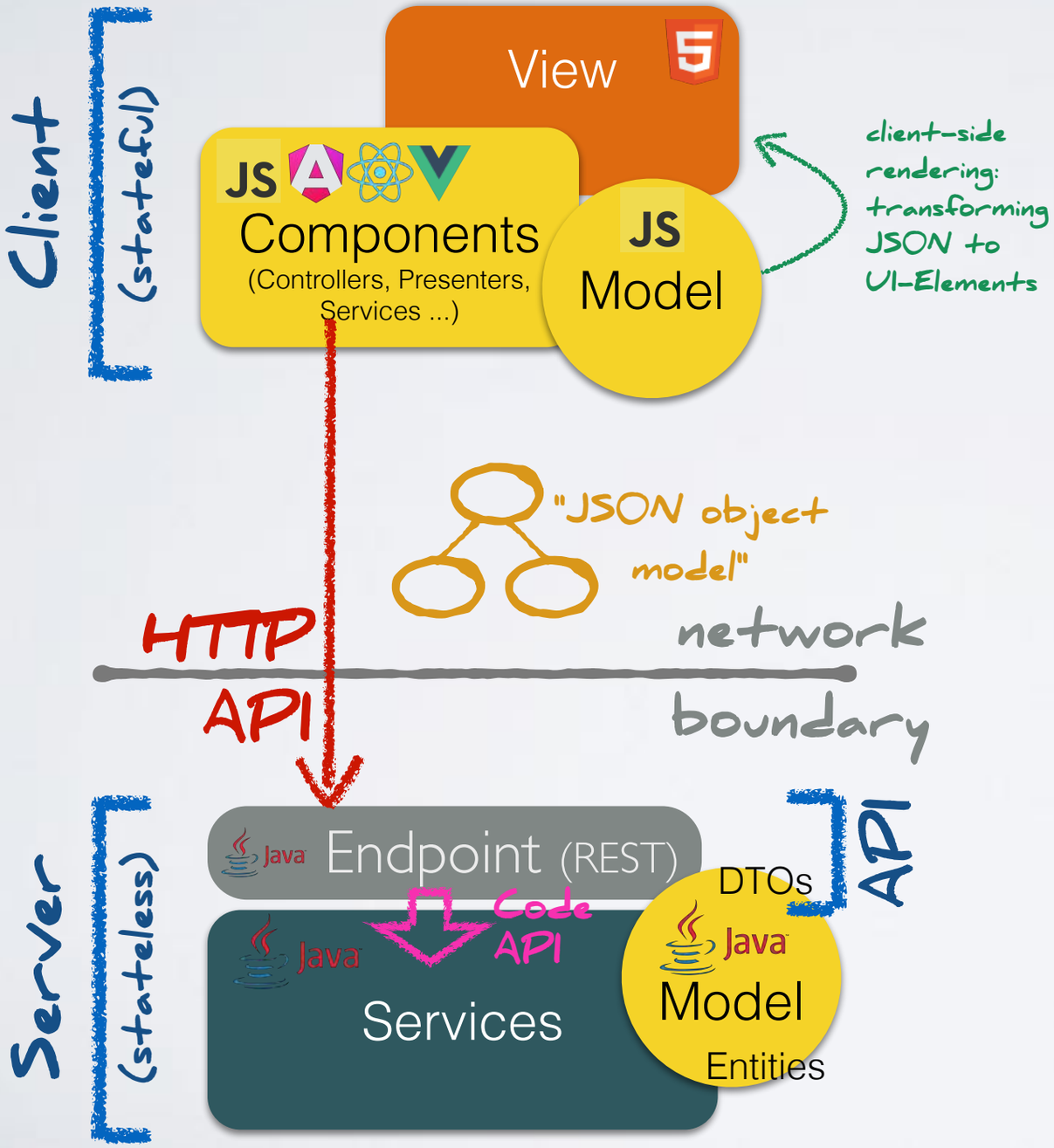
STAR WARS

THE RISE OF THE API

EPISODE IX

The architecture for Single Page Applications implied a HTTP API.

The Role of the API



The rise of SPA development caused a "de-facto" architecture of formalized HTTP/REST-APIs.

Symptoms:

- "API-First" Design
- "The central role of API-Gateways" (the return of ESBs)

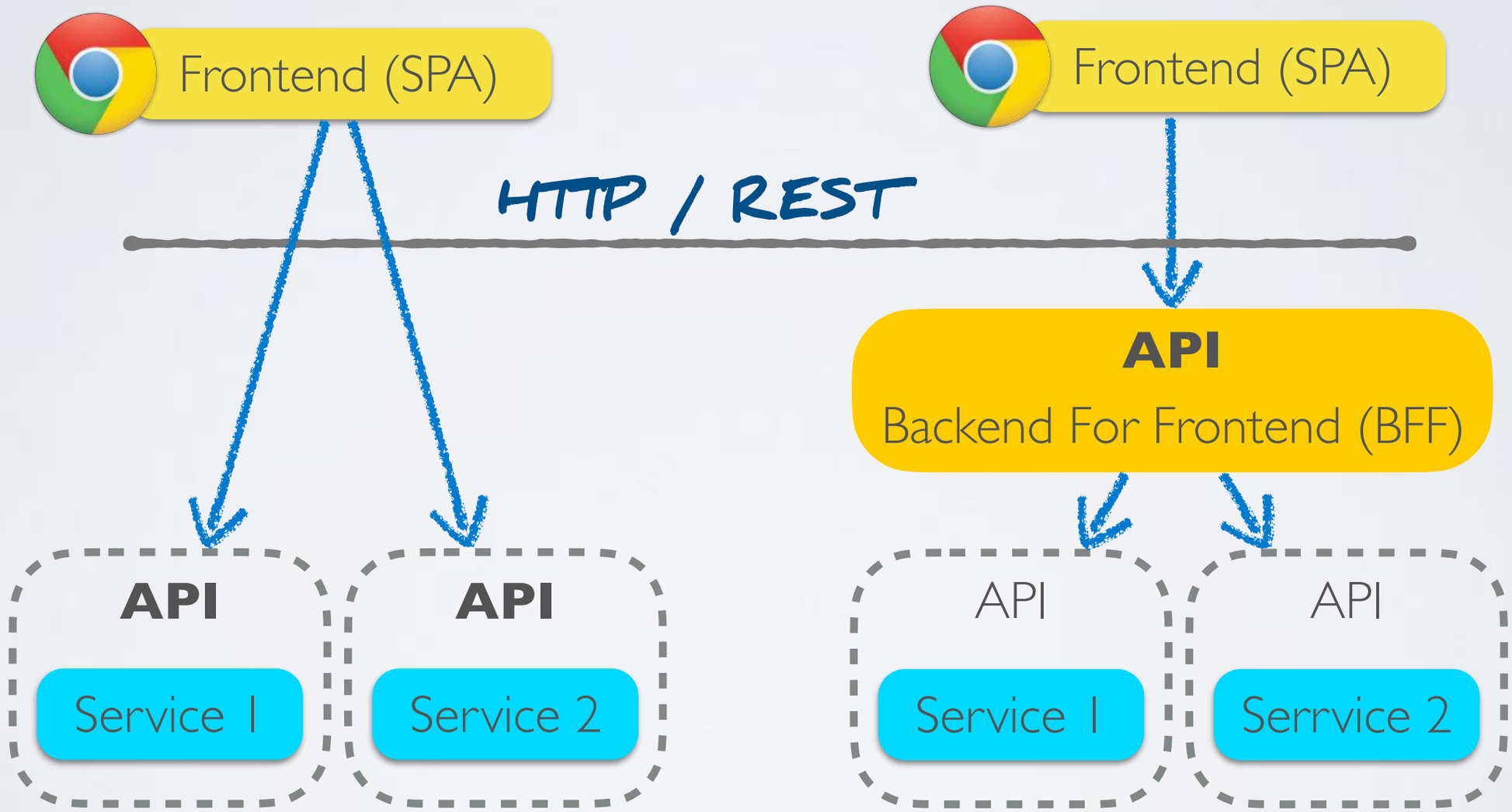
...

Creating a formalized API is a non-trivial effort:
Proper design of URLs, Mapping, Serialization, Security ...

There are advantages in a formalized HTTP API: separation of concern, clearly specified and testable boundaries, reuse, team separation ...

Traditional Architectures for SPAs

Client - API - Server



https://samnewman.io/patterns/architectural/bff/



Components

Component Architecture

Current Frameworks (Angular, React, Vue.js) moved away from the "traditional" MVC pattern to a "Component Architecture".



Components:

- divide & structure
- encapsulation (view & logic)
- reuse
- defined boundaries
- data flow

Typically the whole application itself is a component ("root component"). The html document is just a "shell" in which the application is loaded.

Components

Components in a Web-UI typically consist of two parts:

- a "template" which represents the DOM structure (potentially including styling)
- JavaScript code that implements the behavior

Modern SPA Frameworks like Angular or React deliver components in a JavaScript bundle. The template is compiled to DOM manipulation statements.

Examples:

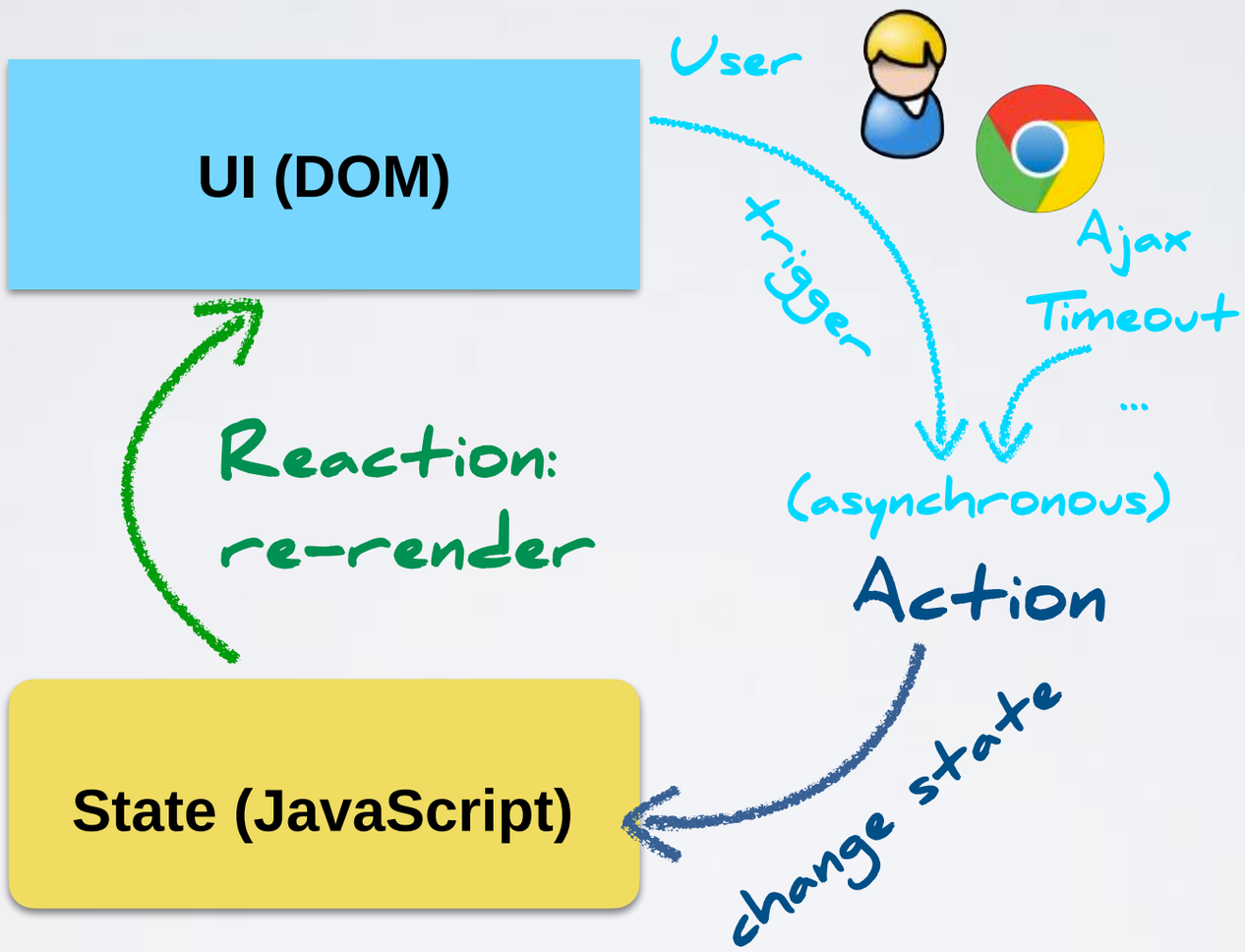
- jQuery template & Handlebars load HTML snippets as text and put them in the DOM ("element.innerHTML")
- In React JSX is compiled to JavaScript
- In Angular and Vue templates are compiled to JavaScript



"Reactivity"

State is Managed in JavaScript

The UI renders the state and "signals" events.



Reactivity in a SPA: The application reacts on state changes and updates the UI.

Evan You - Reactivity in Frontend JavaScript Frameworks: https://www.youtube.com/watch?v=r4pNEdIt_I4
http://teropa.info/blog/2015/03/02/change-and-its-detection-in-javascript-frameworks.html

Reactivity ... What and Why?

Traditional "DOM-centric" applications

UI = state

Browsers have "built-in reactivity": If the DOM is changed, the UI is re-rendered.

With modern SPA architectures (MVC, MVP, Components ...) the client state is represented as JavaScript objects.

UI = f(state)

When to call?

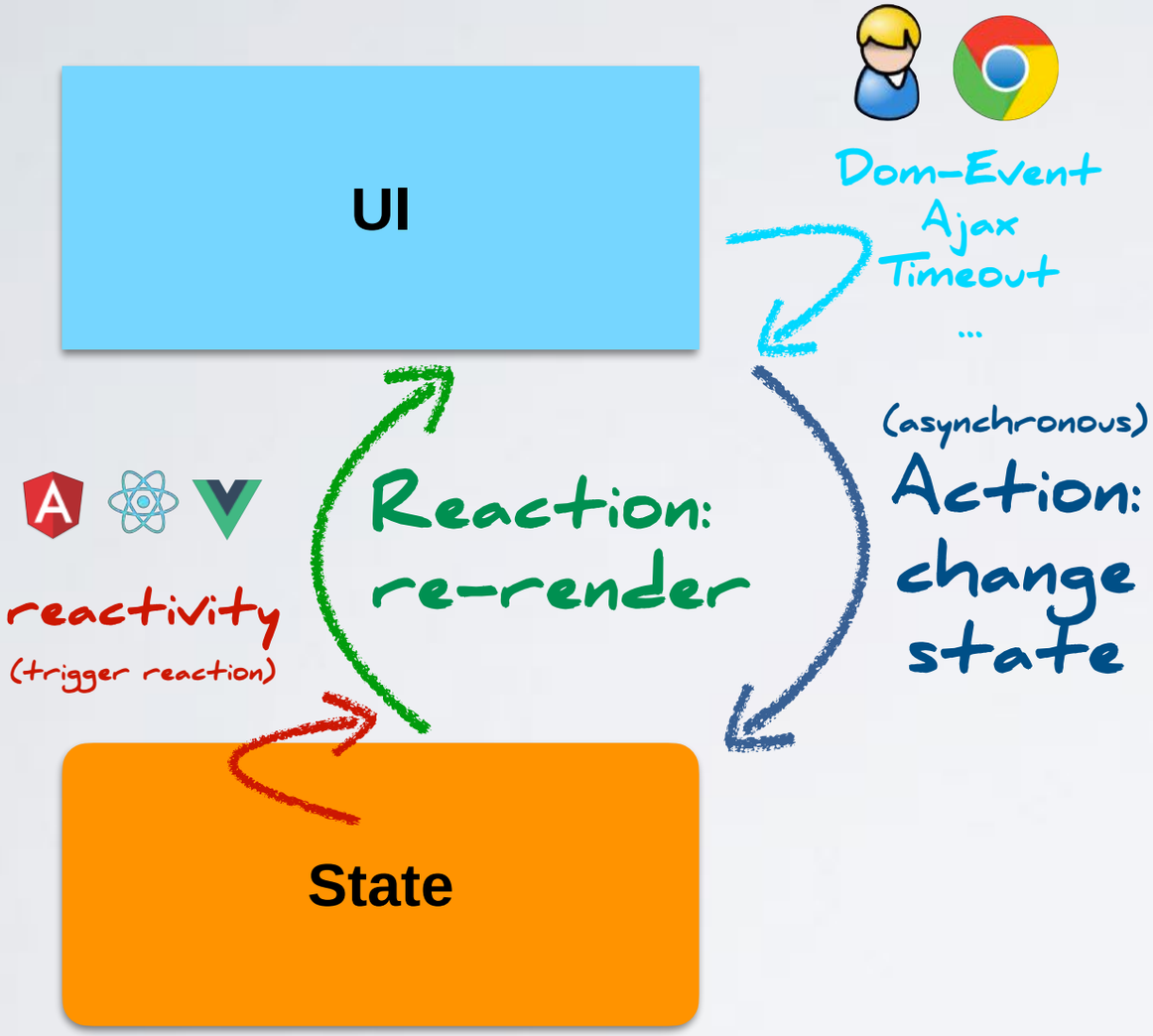
The UI that you can see and manipulate on screen is the result of painting a visual representation of data.

Reactive programming in general addresses the question: How to deal with change over time?

The UI should (automatically) re-render when the state changes.

https://blog.danlew.net/2017/07/27/an-introduction-to-functional-reactive-programming/

Frameworks Reactivity



Front end development and change detection (Angular vs. React):
https://www.youtube.com/watch?v=1i8klHov3vA
https://github.com/in-depth-education/change-detection-in-web-frameworks

https://blog.thoughtram.io/angular/2016/02/22/angular-2-change-detection-explained.html
https://vuejs.org/v2/guide/reactivity.html

Angular: different approachs to Reactivity

Angular comes with *Zone.js*. This is a library that patches the native Browser APIs in order to notify the app when a potential change happens. It triggers the Angular change-detection algorithm which implements dirty checking.
In Angular changes are often explicitly modeled as streams of events/data with RxJS Observables.
With signals Angular offers another API to explicitly model reactive state.
Future versions of Angular will be zoneless and offer fine grained reactivity based on reactive state.

React: Reactivity is based on `useState` / `setState`

`useState` triggers a complete rendering of the component and its child components into the virtual DOM (this can be optimized).

Vue: Reactive state base on `ref()` and `reactive()` (basically signals)

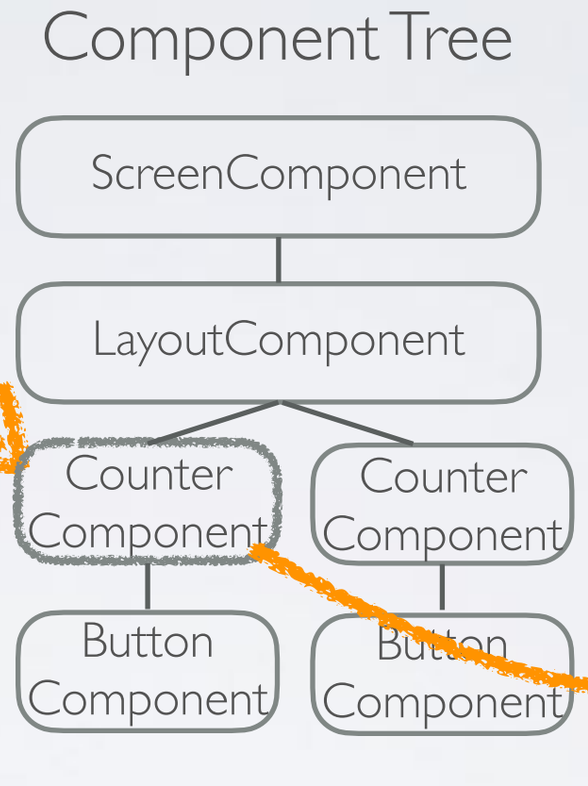
Vue transparently converts properties to *getter/setters*. This allows dependency-tracking and change-notification.

Fine Grained Reactivity

"optimized re-rendering on state changes"

```
setInterval(
    () => this.count.set(
      this.count() + 1;
    )
1000);
```

trigger re-rendering



DOM

```
...
<div>
 Count 1:
 <span>42</span>
 <button>Increase</button>
<div>
...
```

Fine grained reactive state only triggers re-rendering on components (or blocks) that depend on changed state.

Many frameworks already implement (more or less) fine grained reactivity: Solid, Qwik, Svelte, Vue ...

Client-Side Routing in a SPA



http://thedomain:80/app/first



Browser



Server

initial request
http://thedomain:80/app



html-shell is loaded



"navigate" to "first" screen



load data &
render screen with JS



http://thedomain:80/app/**first**



navigation



"navigate" to
"second" screen



load data &
render screen with JS



http://thedomain:80/app/**second**

html-shell



assets



optional ajax request
http://thedomain:80/api/data/first

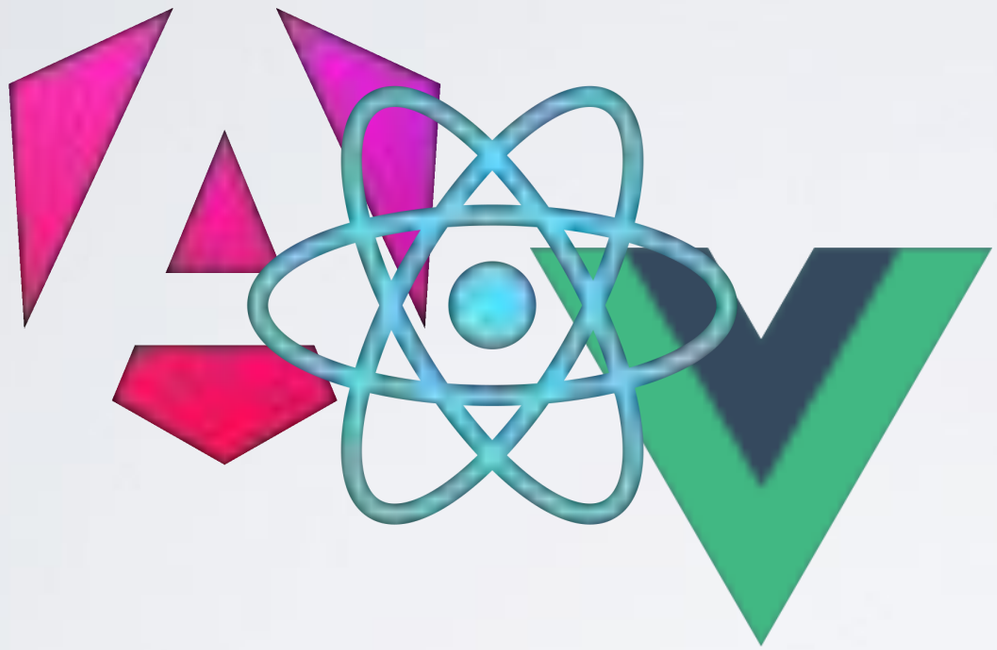
data

optional ajax request
http://thedomain:80/api/data/second

data

app bootstrapping

Server Side Rendering (SSR)



Today every modern frontend framework is capable of server side rendering.

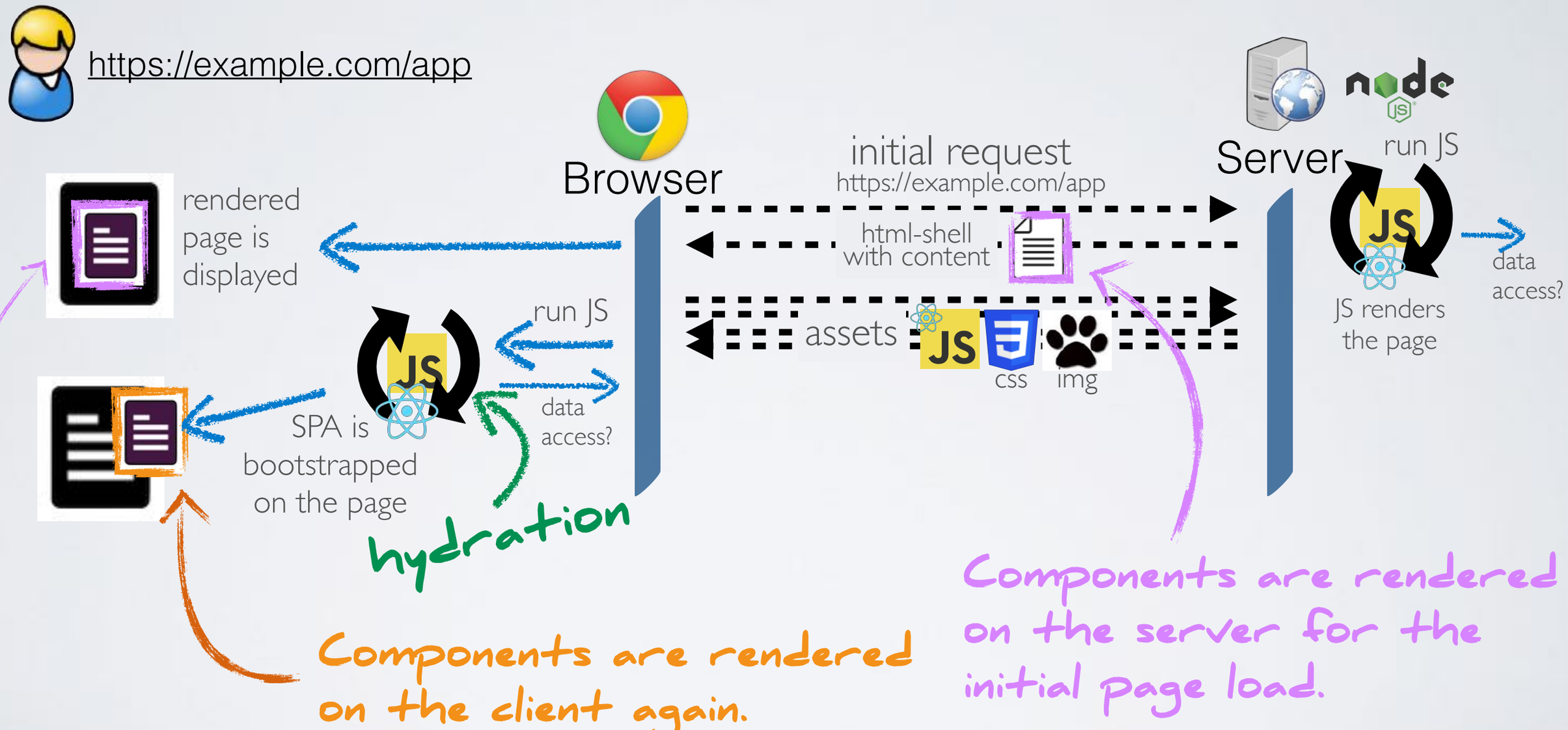
"Production-grade SSR" goes beyond rendering components to a html-string. This requires deeper integrations into bundling as well as data-fetching and routing.

- In React and Vue this one of the reason for "Meta-Framework": Next.js, Remix/ReactRouter v7, Nuxt, Quasar ...
- Angular is capable of SSR "out-of-the-box" since v17 and they are improving SSR in every version.

SPA with Server Side Rendering (SSR)

(*initial* rendering on the server - *hydration* on the client)

Analogy to a "game engine" where the first frame is rendered on the server and every subequent frame is rendered on the client.



Advantages:

- search indexing / social previews
- improving time to first contentful paint

SSR has its own challenges:

- UX (page is not interactive on first render)
- Data Access (different mechanisms on the client and server)
- Browser APIs (not available on the server)
- rendering differences between client and server & hydration errors

"Full-Stack" Frameworks

NEXT.JS

React Router
Remix
v7

RedwoodJS

TanStack Start

Nuxt

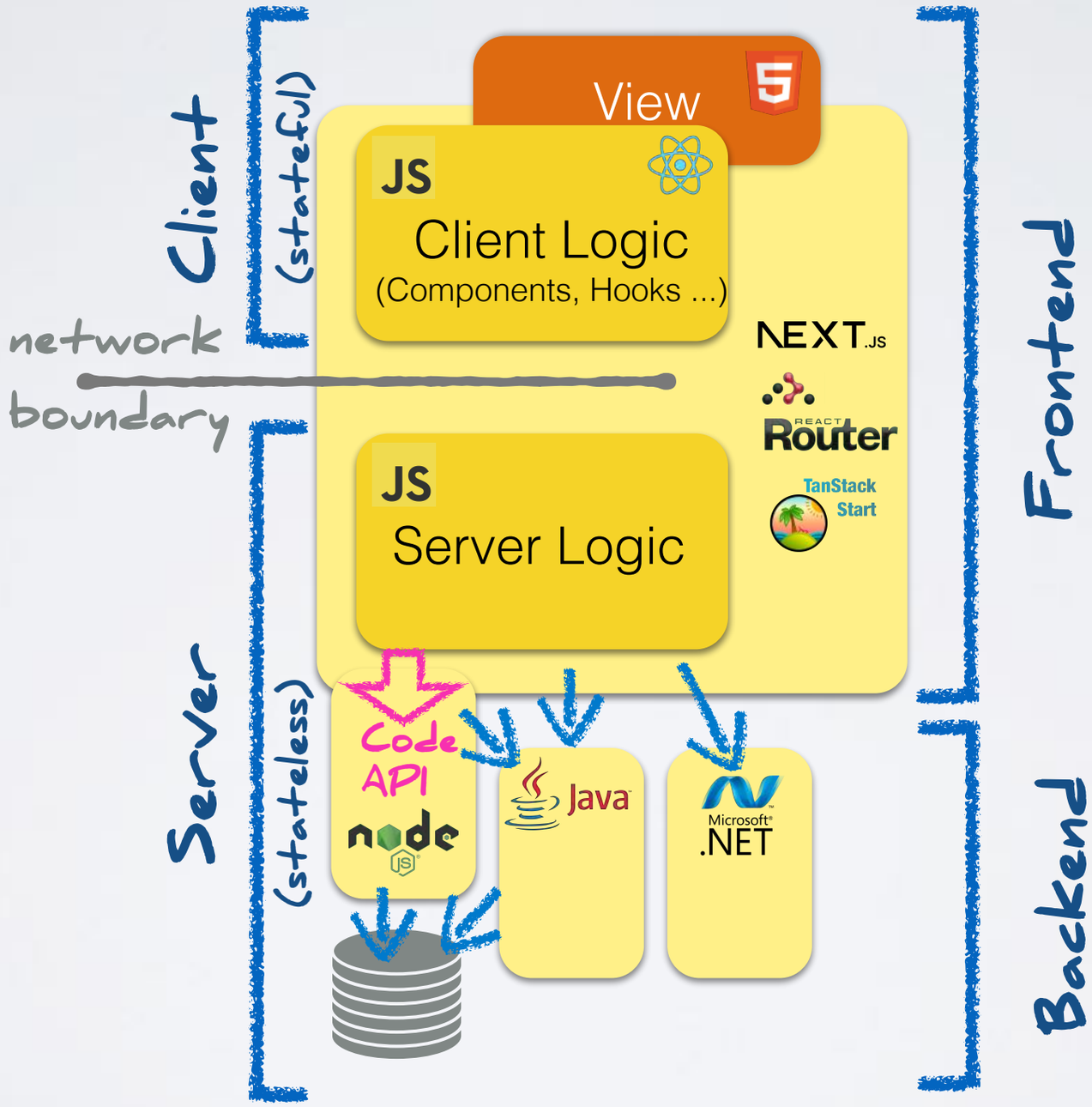
Analog

SOLIDSTART

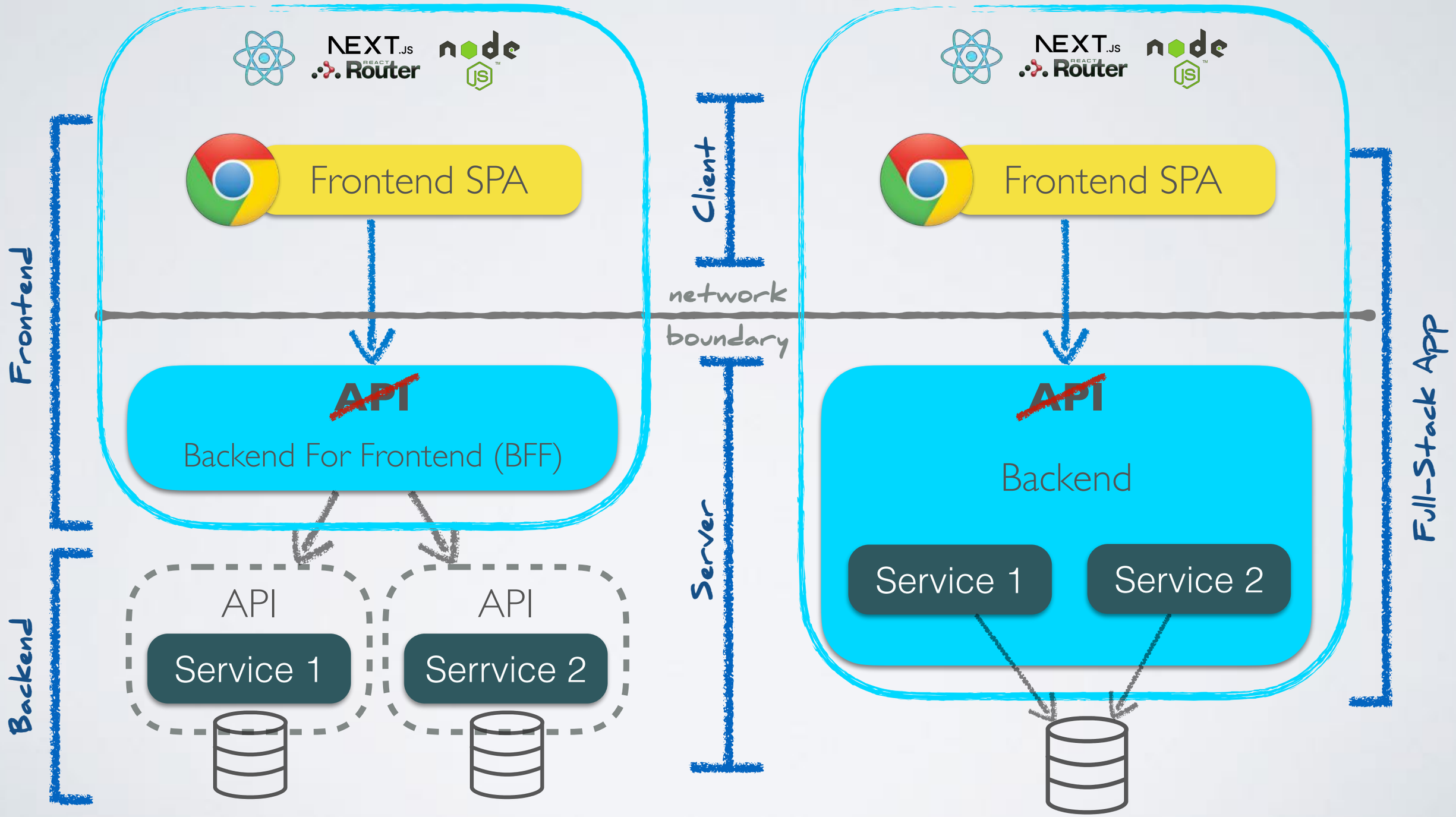
SVELTEKIT

qwik Qwik City

Full-Stack Frontend Frameworks

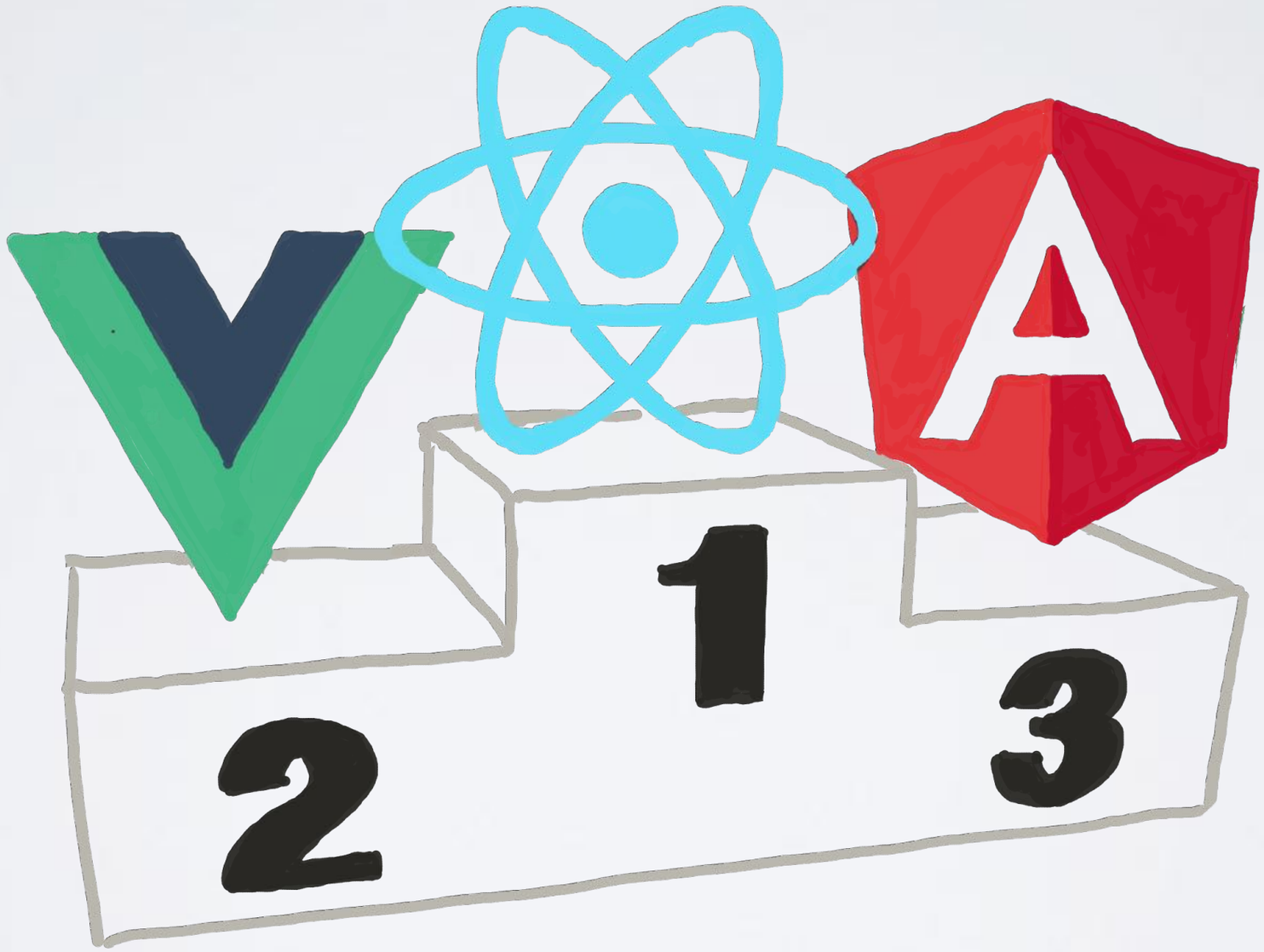


React Full-Stack Architectures



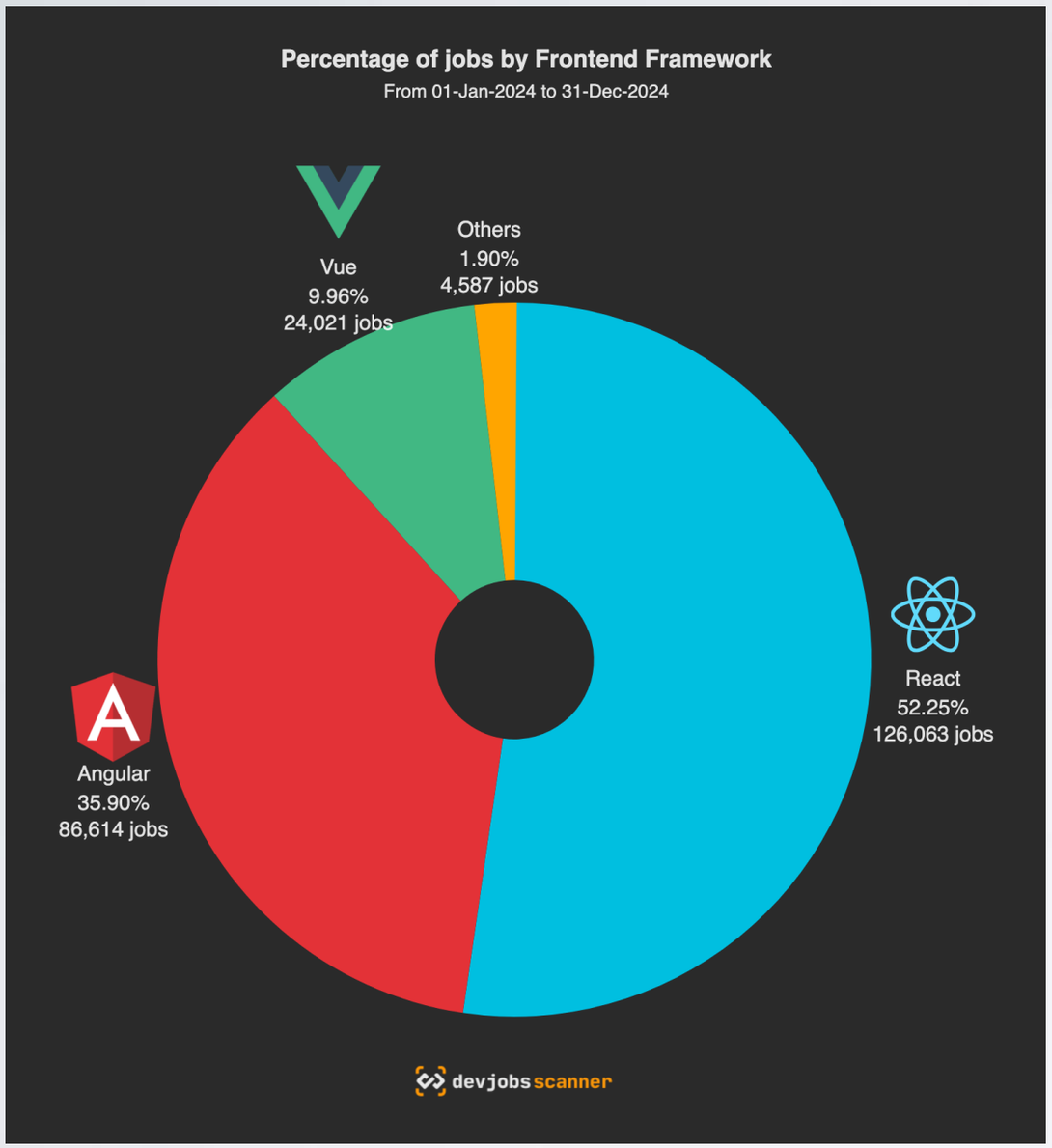
https://samnewman.io/patterns/architectural/bff/

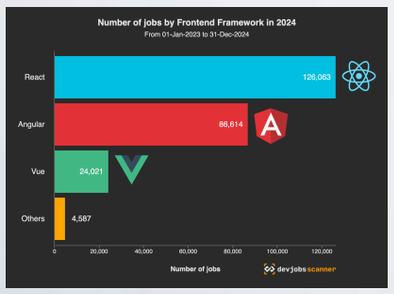
Which one is The Best™ ?

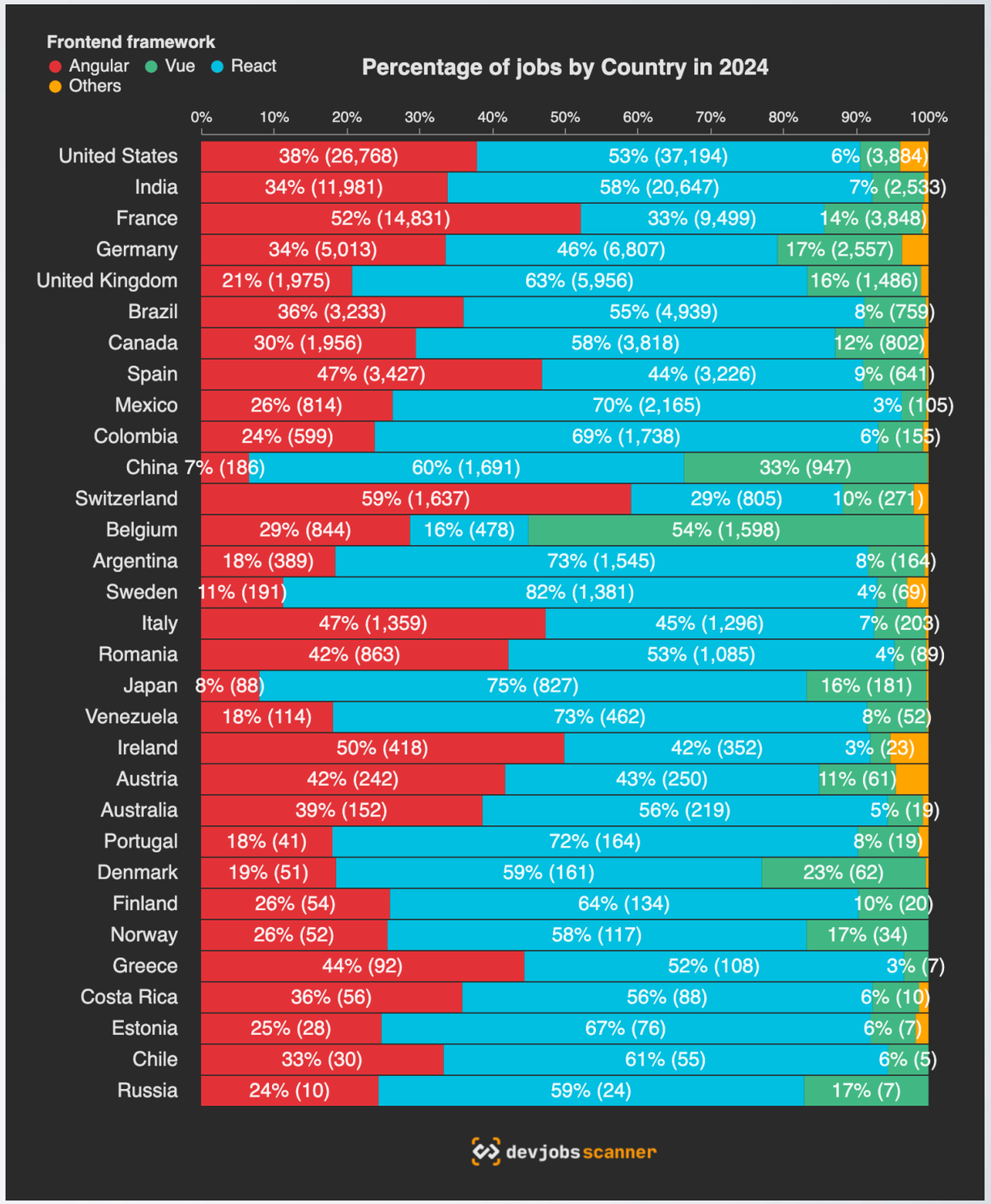


https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/

Popularity: Angular vs. React vs. Vue

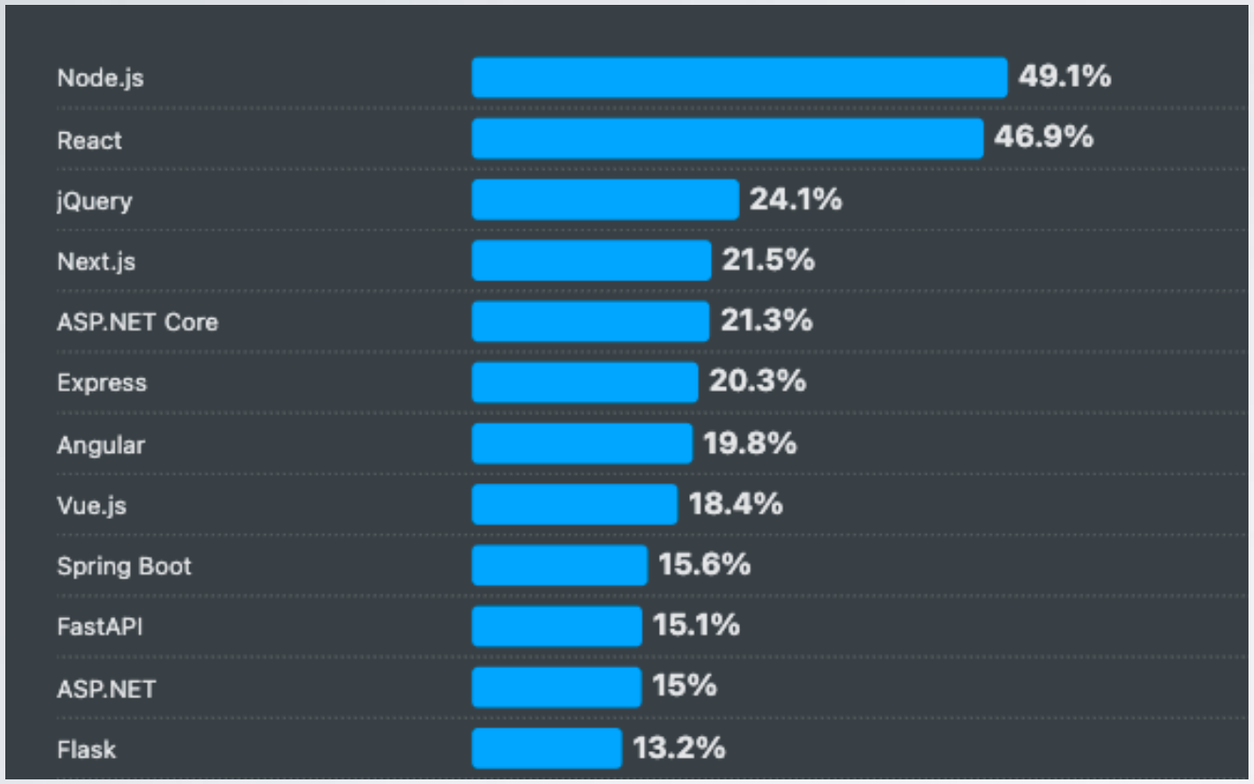






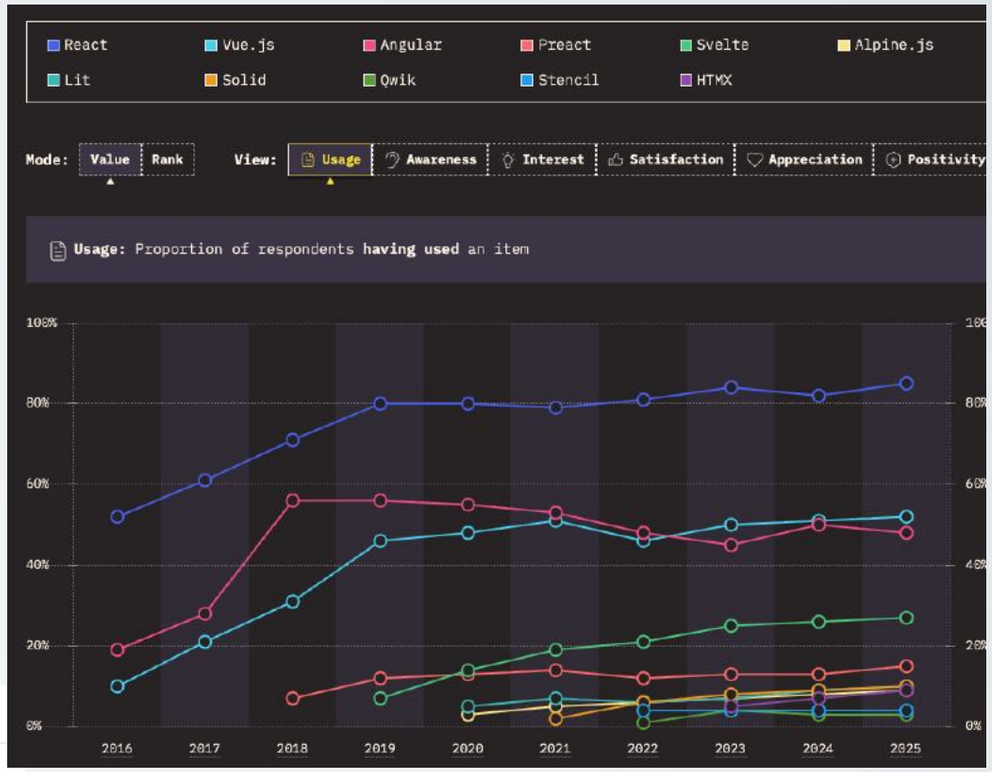
https://www.devjobsscanner.com/blog/the-most-demanded-frontend-frameworks/

Popularity: Angular vs. React vs. Vue

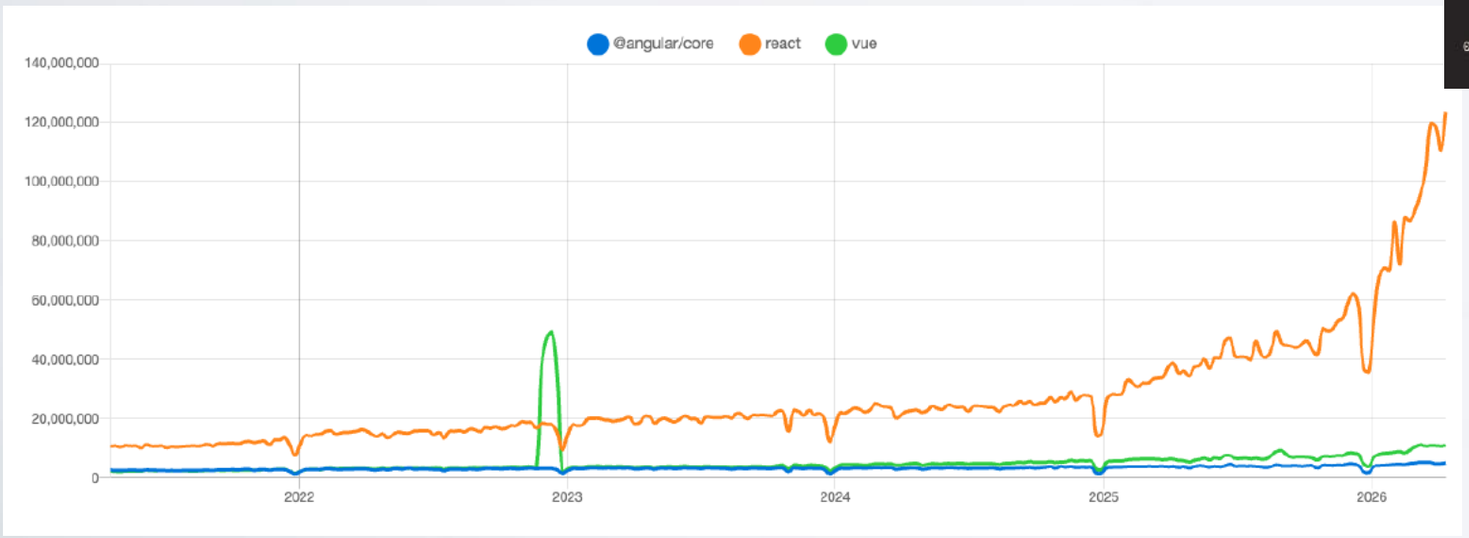


https://survey.stackoverflow.co/2025/technology#1-web-frameworks-and-technologies

State of JavaScript Survey 2025



https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/



https://npmtrends.com/@angular/core-vs-react-vs-vue

Framework usages of Angular, React and Vue are all still growing.

At the moment it does not look like one of the frameworks is "killing" another framework.

Performance Comparison

Name Duration for...	vue-vapor-v3.6.0-alpha.2	solid-v1.9.3	svelte-v5.42.1	vue-v3.6.0-alpha.2	angular-cf-nozone-v21.0.5	react-hooks-v19.2.0	react-mlyn-v0.5.16	angular-cf-v21.0.5	react-tagged-state-v19.0.0 + 2.1.0	react-com-piler-hooks-v19.0.0	angular-cf-signals-nozone-v21.0.5	react-kr-observ-able-v19.0.0 + 3.0.8	react-zus-tand-v19.0.0 + 5.0.2	react-mobX-v19.0.0 + 6.13.5	angular-cf-signals-v21.0.5	angular-ngfor-v21.0.5	react-tracked-v19.0.0 + 2.0.1
Implementation notes							1139										
Implementation link	code	code	code	code	code	code	code	code	code	code	code	code	code	code	code	code	code
Language	JavaScript	JavaScript	JavaScript	JavaScript	TypeScript	JavaScript	JavaScript	TypeScript	JavaScript	JavaScript	TypeScript	JavaScript	JavaScript	JavaScript	TypeScript	TypeScript	JavaScript
create rows creating 1,000 rows. (5 warmup runs).	23.2 ±0.2 (1.05)	23.0 ±0.1 (1.04)	23.1 ±0.1 (1.04)	26.5 ±0.1 (1.19)	30.0 ±0.2 (1.35)	27.4 ±0.2 (1.23)	29.3 ±0.4 (1.32)	37.1 ±0.4 (1.67)	27.9 ±0.4 (1.26)	27.6 ±0.1 (1.24)	35.6 ±0.3 (1.60)	28.2 ±0.3 (1.27)	28.8 ±0.2 (1.30)	29.7 ±0.3 (1.34)	37.3 ±0.5 (1.68)	37.2 ±0.7 (1.68)	29.5 ±0.5 (1.33)
replace all rows updating all 1,000 rows. (5 warmup runs).	25.9 ±0.2 (1.07)	26.1 ±0.2 (1.08)	26.6 ±0.1 (1.10)	29.4 ±0.2 (1.22)	34.7 ±0.1 (1.44)	32.4 ±0.3 (1.34)	33.1 ±0.2 (1.37)	39.5 ±1.1 (1.64)	33.3 ±0.4 (1.38)	32.3 ±0.3 (1.34)	38.3 ±0.9 (1.59)	34.5 ±0.3 (1.43)	32.8 ±0.2 (1.36)	36.4 ±0.4 (1.51)	41.8 ±1.6 (1.73)	38.9 ±0.9 (1.61)	35.2 ±0.3 (1.46)
partial update updating every 10th row for 1,000 row. (3 warmup runs). 4 x CPU slowdown.	10.9 ±0.2 (1.15)	10.7 ±0.2 (1.13)	10.7 ±0.3 (1.13)	12.8 ±0.2 (1.35)	11.5 ±0.2 (1.21)	13.6 ±0.4 (1.43)	14.4 ±0.3 (1.52)	11.8 ±2.4 (1.24)	14.2 ±0.3 (1.49)	13.9 ±0.2 (1.46)	12.4 ±2.5 (1.31)	14.7 ±0.4 (1.55)	15.6 ±0.3 (1.64)	15.6 ±0.3 (1.64)	12.5 ±1.9 (1.32)	11.3 ±2.2 (1.19)	20.2 ±0.3 (2.13)
select row highlighting a selected row. (5 warmup runs). 4 x CPU slowdown.	2.5 ±0.2 (1.14)	3.1 ±0.1 (1.41)	4.7 ±0.3 (2.14)	3.4 ±0.3 (1.55)	5.8 ±0.2 (2.64)	4.3 ±0.2 (1.95)	5.4 ±0.2 (2.45)	5.7 ±0.2 (2.59)	5.1 ±0.2 (2.32)	6.5 ±0.2 (2.95)	6.1 ±0.2 (2.77)	5.0 ±0.2 (2.27)	4.9 ±0.3 (2.23)	5.8 ±0.1 (2.64)	6.2 ±0.2 (2.82)	5.6 ±0.2 (2.55)	9.6 ±0.1 (4.36)
swap rows swap 2 rows for table with 1,000 rows. (5 warmup runs). 4 x CPU slowdown.	12.1 ±0.2 (1.06)	13.1 ±0.3 (1.15)	13.1 ±0.4 (1.15)	13.2 ±0.3 (1.16)	14.8 ±0.4 (1.30)	96.4 ±0.8 (8.46)	97.1 ±0.8 (8.52)	14.7 ±2.6 (1.29)	95.1 ±0.8 (8.34)	95.8 ±0.7 (8.40)	23.9 ±1.8 (2.10)	96.8 ±0.6 (8.49)	97.0 ±1.0 (8.51)	98.4 ±0.9 (8.63)	24.0 ±2.4 (2.11)	122.4 ±2.3 (10.74)	98.3 ±0.6 (8.62)
remove row removing one row. (5 warmup runs). 2 x CPU slowdown.	9.8 ±0.1 (1.05)	9.9 ±0.1 (1.06)	10.0 ±0.1 (1.08)	11.9 ±0.1 (1.28)	11.6 ±0.1 (1.25)	11.0 ±0.1 (1.18)	11.2 ±0.3 (1.20)	10.2 ±1.1 (1.10)	10.9 ±0.2 (1.17)	11.6 ±0.2 (1.25)	11.9 ±0.9 (1.28)	11.3 ±0.3 (1.22)	11.9 ±0.2 (1.28)	11.1 ±0.2 (1.19)	14.7 ±0.7 (1.58)	12.1 ±1.1 (1.30)	12.7 ±0.2 (1.37)
create many rows creating 10,000 rows. (5 warmup runs).	249.3 ±0.4 (1.10)	245.5 ±0.3 (1.09)	247.2 ±0.5 (1.09)	275.7 ±0.4 (1.22)	328.7 ±1.0 (1.45)	378.4 ±1.6 (1.67)	402.7 ±2.4 (1.78)	337.8 ±1.2 (1.49)	381.2 ±0.8 (1.69)	379.4 ±0.9 (1.68)	330.5 ±1.2 (1.46)	394.0 ±1.8 (1.74)	391.7 ±1.3 (1.73)	404.7 ±4.0 (1.79)	338.6 ±1.6 (1.50)	341.0 ±1.2 (1.51)	389.3 ±1.1 (1.72)
append rows to large table appending 1,000 to a table of 1,000 rows. (5 warmup runs).	27.4 ±0.3 (1.09)	26.5 ±0.3 (1.06)	26.8 ±0.2 (1.07)	29.6 ±0.1 (1.18)	34.6 ±0.1 (1.38)	32.0 ±0.2 (1.27)	34.8 ±0.2 (1.39)	37.3 ±0.5 (1.49)	33.4 ±0.2 (1.33)	32.4 ±0.2 (1.29)	38.0 ±0.6 (1.51)	33.3 ±0.1 (1.33)	33.6 ±0.1 (1.34)	34.9 ±0.1 (1.39)	37.2 ±0.4 (1.48)	36.7 ±1.1 (1.46)	34.8 ±0.2 (1.39)
clear rows																	

https://krausest.github.io/js-framework-benchmark/current.html

Conclusion:

- Angular, React and Vue are very close when it comes to framework performance.
- The differences are not significant for typical "line of business" applications.
- There are many modern frameworks that are faster than Angular, React (and Vue).

Comparison

Templating

templates: enhancing html with dynamic constructs

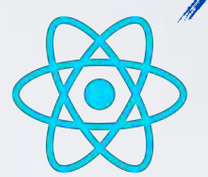


separate files or inlined strings



single file components

(Vue also supports render functions with JSX)



JSX: declaring UI with code

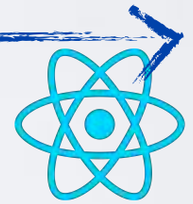
Framework vs Library



"complete" framework with opinionated choices



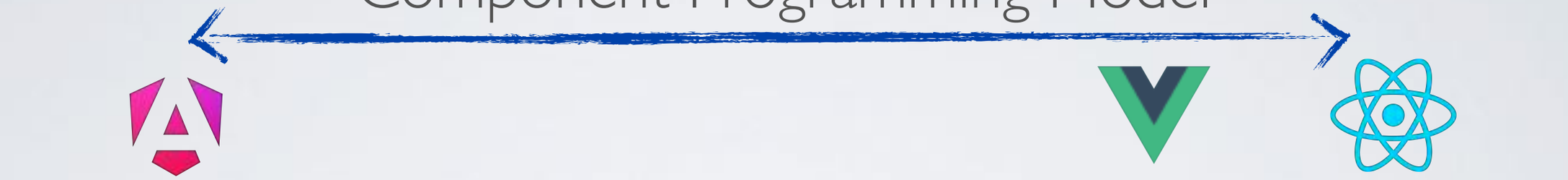
small & unopinionated library providing default options for important topics (router, centralized state management)



small & unopinionated library that must be combined with other libraries

Comparison

Component Programming Model

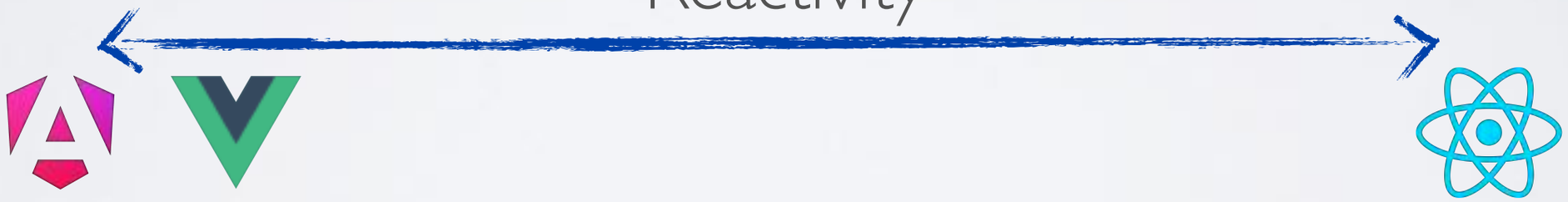


based on classes

functional API (composition API)

new functional API ("Hooks")

Reactivity



fine grained reactivity with signals

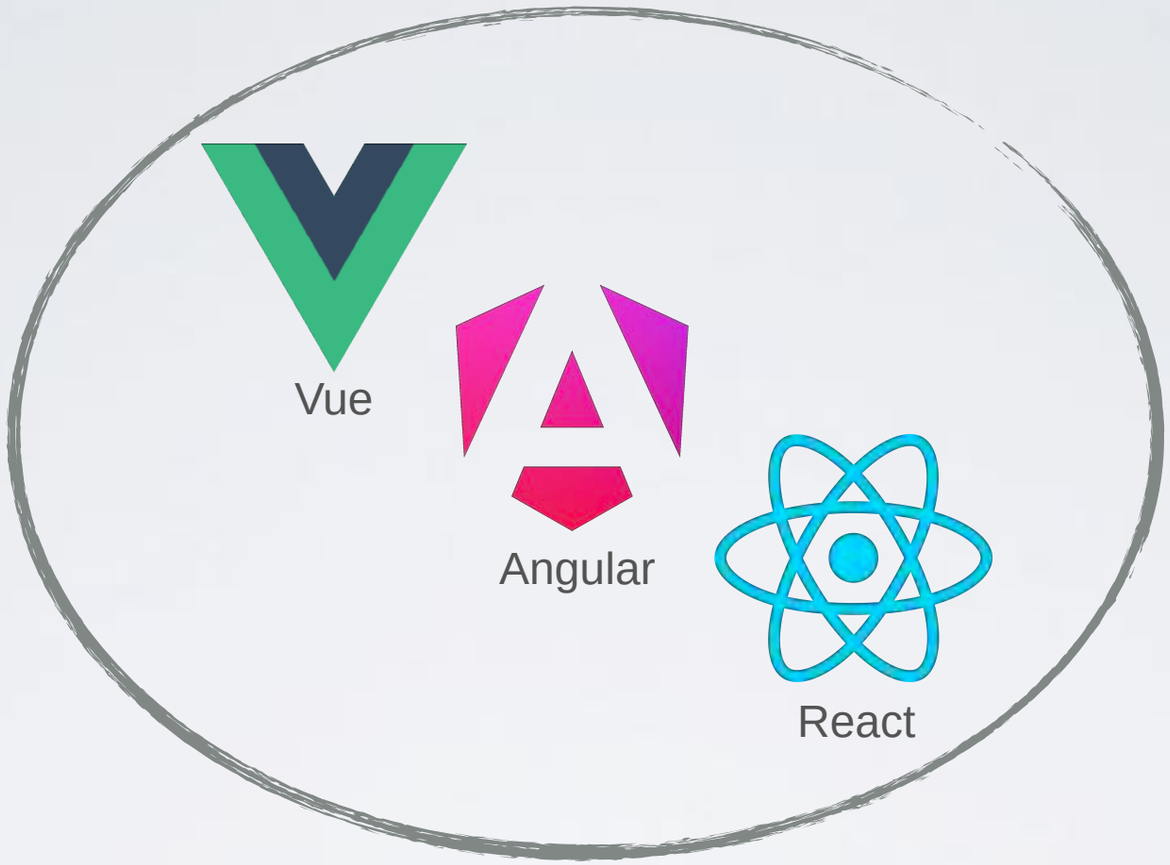
coarse grained render functions
(optimized by the React Compiler)

- Angular tries to be the "stable and reliable" framework. But right now it is in a phase of heavy changes.

- React is the wild west:
  You have to choose a framework on top of React and/or choose & maintain a stack of libraries.

- Vue is currently the most stable framework and many innovations came out of the Vue ecosystem.

Typical Single Page Applications



Angular, React and Vue Projects are very similar:

- based on modern JavaScript (incl. TypeScript support)
- built on top of the npm ecosystem
- require Node.js & npm tooling at build-time
- heavy conceptual separation of frontend and backend
- "unopinionated" about backend technologies
- embracing the Browser as "application platform"

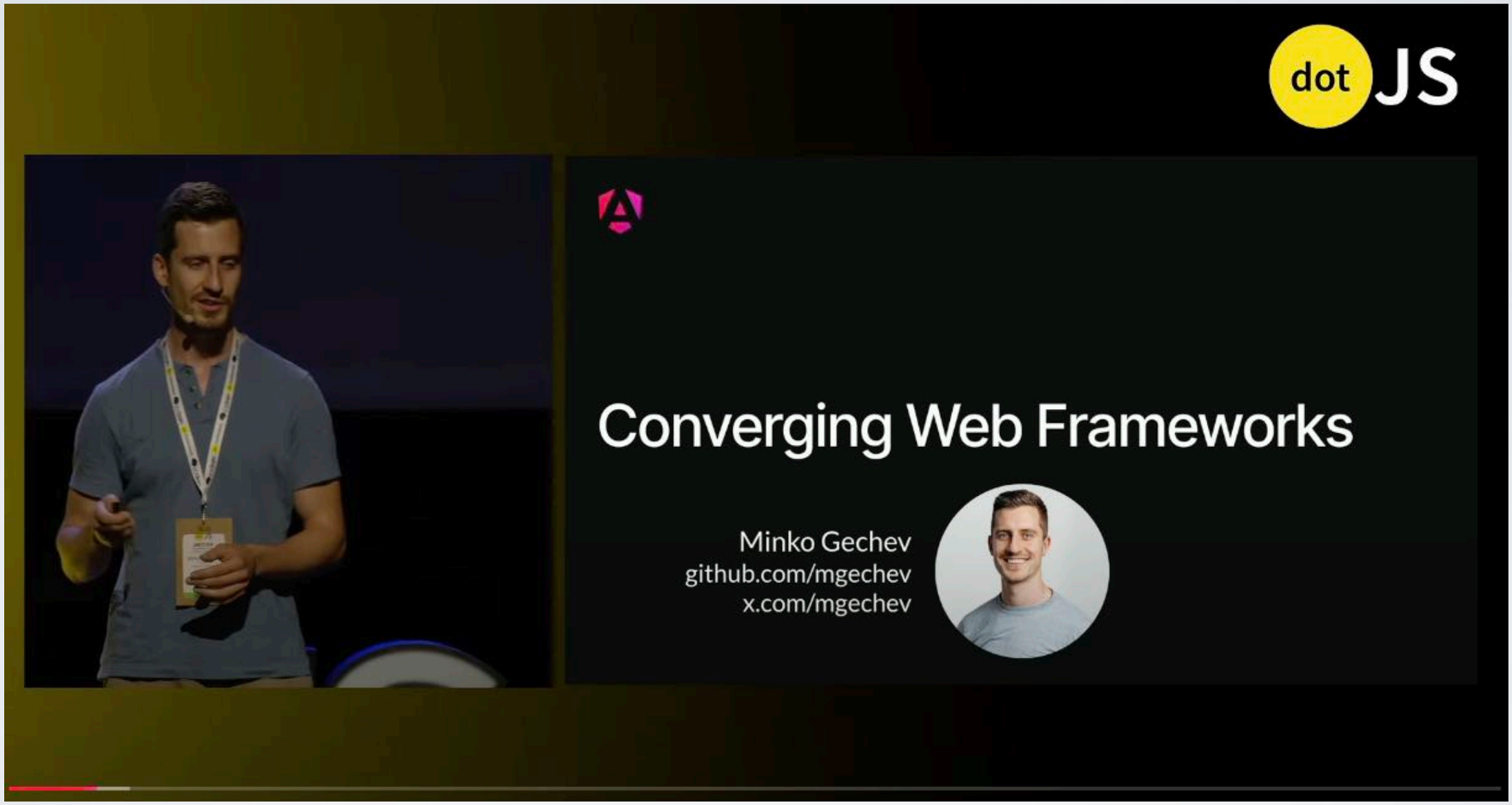
Does it even matter?

Angular, React and Vue are very similar.

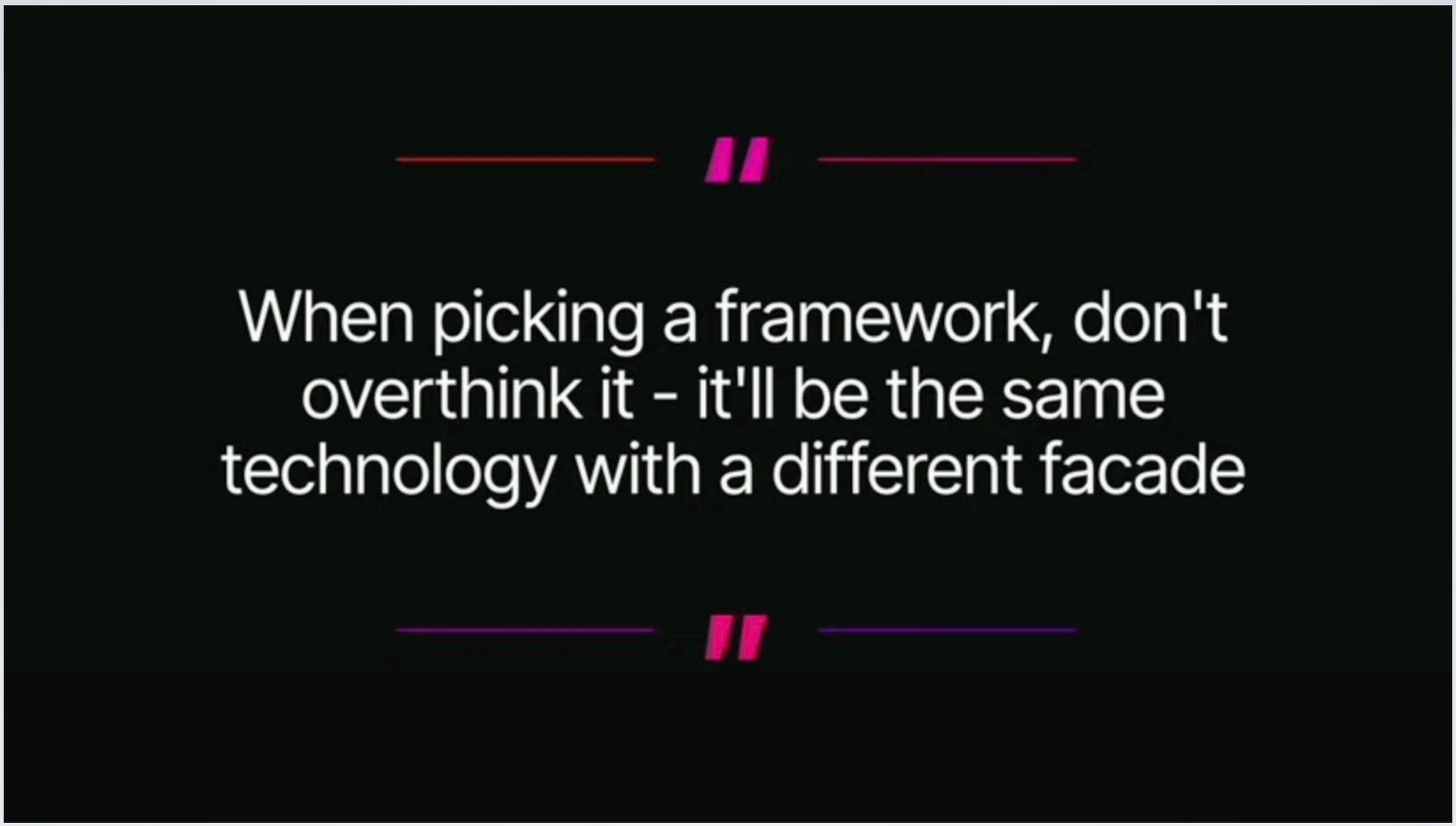
Choosing the flavor of "state-of-the-art" SPA frontend framework is *not a critical success factor* for your project.
Architecture, tooling, development process and organizational factors have *an order of magnitude more influence* on the success of your project.

Thesis:

Current Frameworks have become very similar



dotJS 2024 - Minko Gechev - Product Lead for Angular at Google
Converging Web Frameworks: https://www.youtube.com/watch?v=grRH8e46Pso



dotJS 2024 - Minko Gechev - Product Lead for Angular at Google
Converging Web Frameworks: https://www.youtube.com/watch?v=grRH8e46Pso

Consider non-technical factors:
Stability & Reliability, Community, Enjoyment

Framework Convergence by Example



Signals for fine grained reactivity

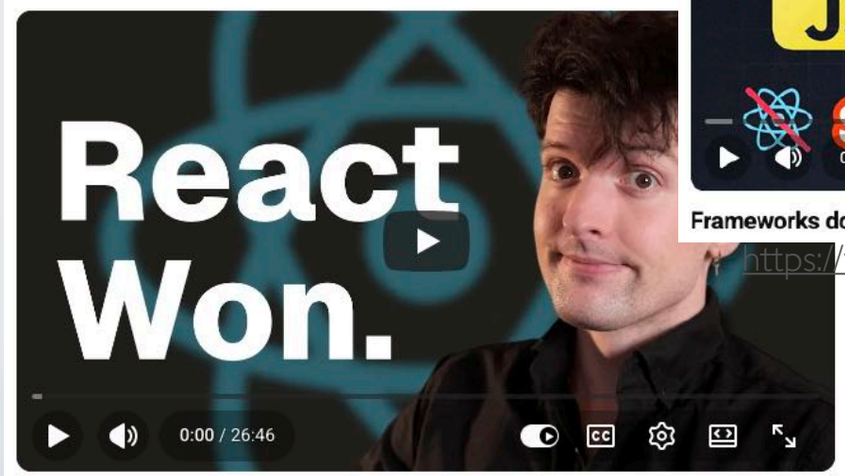


Build Tooling based on Vite

JS Frameworks in the Age of AI

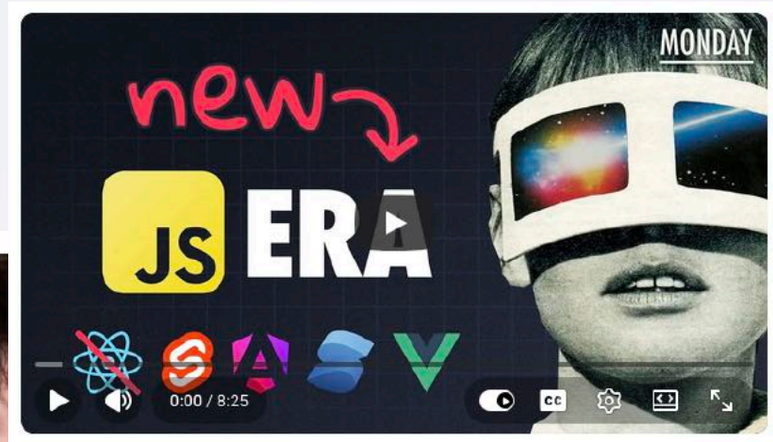
React offers the largest taining-set for AI and therefore LLMs typically choose React when creating web applications.

- However React offers also the greatest taining set for deprecated code and concepts ...
- How do we get innovation, if new frameworks/concepts don't stand a chance?



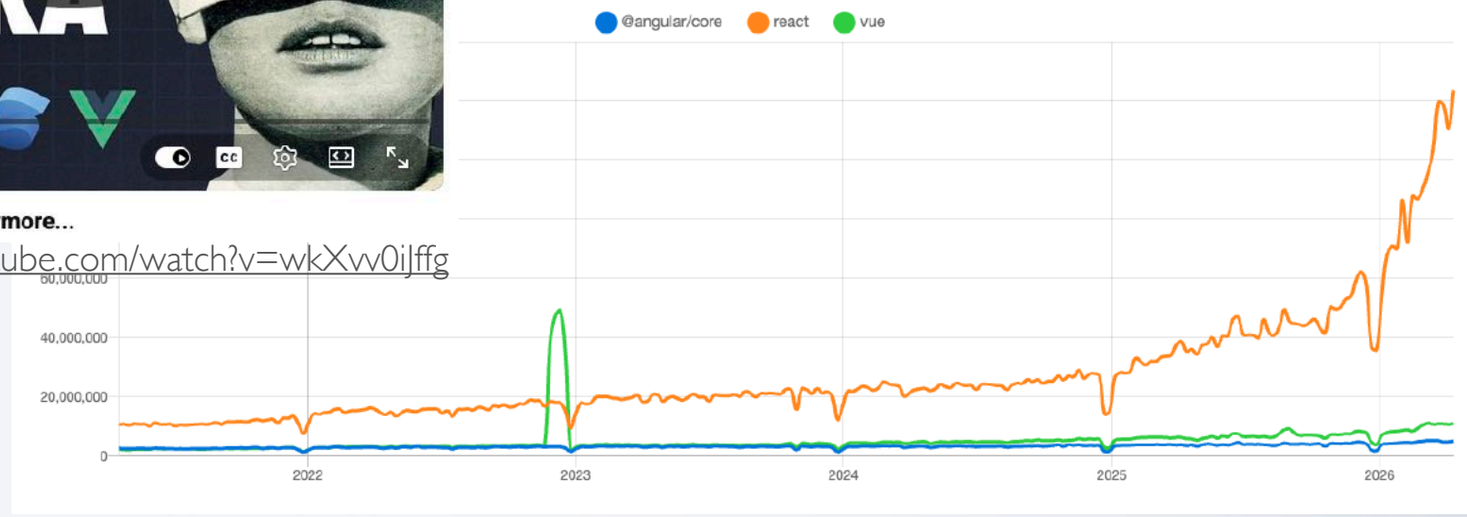
React Is the last framework.

https://www.youtube.com/watch?v=P1FLEnKZTAE



Frameworks don't matter anymore...

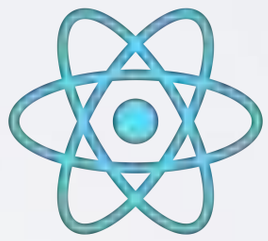
https://www.youtube.com/watch?v=wkXw0iJffg



https://npmtrends.com/@angular/core-vs-react-vs-vue

Who do you (want to) trust?

Google 

facebook 

Open Source Community 

(with many financial sponsors)



Styling Mechanisms

The ecosystem of tooling around styling frontend applications is chaotic, especially in the React ecosystem!

Today styling it is very much entangled with frontend frameworks and component libraries.

Concerns

Scoping

Colocation

Developer Experience

Performance

Dynamic Styling

Angular, Vue, Svelte offer an integrated styling mechanism.
React, Solid, Preact do not offer a specific styling mechanism.

Styling Philosophies



Component should not be styled

"The design of the components should be separate from their implementation."

Component should be styled

"Components should be a higher level of abstraction and solve the topic of styling (design systems)"

History

CSS Frameworks: Bootstrap, Bulma

- https://getbootstrap.com/
- https://bulma.io/

Critique: missing customizability

Conceptually not aligned with abstractions of component-based frameworks.

Consequence:

- https://react-bootstrap.netlify.app/
- https://ng-bootstrap.github.io/#/home
- https://valor-software.com/ngx-bootstrap/

Naming Conventions: BEMas a popular example (Block-Element-Modifier)

https://en.bem.info/

```
<div class="card">
  <img class="card__image" src="image.jpg" alt="Card image" />
  <div class="card__body">
    <h2 class="card__title">Card Title</h2>
    <p class="card__description">This is a short description of the card content.</p>
    <button class="card__button card__button--primary">Learn More</button>
  </div>
</div>
```

CSS Preprocessors

"CSS with superpowers"

SASS: Syntactically Awesome Style Sheets

https://sass-lang.com/



Alternatives: less https://lesscss.org/, stylus https://stylus-lang.com/

CSS Preprocessors have first-level integration in Vue and Angular projects:
https://vuejs.org/api/sfc-spec.html#pre-processors
https://angular.dev/guide/components/styling

However: CSS added many powerful features in recent years, that make preprocessors less needed (i.e. variables, nesting ...)

Main Features of Sass

using variables
Splitting into multiple files
Nested declarations

Native modern CSS constructs:

- custom properties: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_cascading_variables/Using_CSS_custom_properties
- CSS @import https://developer.mozilla.org/en-US/docs/Web/CSS/@import
- Native CSS nesting https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_nesting/Using_CSS_nesting

CSS
MODULES

CSS Modules

https://github.com/css-modules/css-modules/

Expose scoped CSS class names to JavaScript

(can also be combined with sass)

Mostly usded in JSX-based frameworks.

```
import styles from './Greeter.module.css';

export default function Greeter() {
  return (
    <div>
      <h1 className={styles.title}>Styled with CSS module</h1>
    </div>
  )
}
```

Implemented in bundlers Vite, Parcel, Rspack, Turbopack, Bun ...
https://github.com/css-modules/css-modules/blob/master/docs/get-started.md

Vue implements CSS modules in Single File Components:
https://vuejs.org/api/sfc-css-features#css-modules

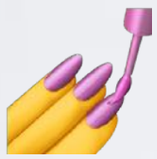
History: CSS-in-JS

(React and JSX-based Frameworks)

CSS-in-JS was the idea to use JS to "define" the styling, like JSX for templating.

not popular any more

This was very popular in the React ecosystem ~2016-2022.
Today its rather a niche/legacy approach.
Several reasons lead to a quick demise of CSS-in-JS: general SSR, React Server Components, the rise of Tailwind ...



styled-components: in maintenance mode since march 2025
https://opencollective.com/styled-components/updates/thank-you



emotion: no support for Next.js since 2022
https://emotion.sh/docs/introduction

Many others: styleX, vanilla-extract, pigmentcss, Panda CSS, Linaria, tss-react, styled-jsx, jss ...

support and project state often unclear ...

https://nextjs.org/docs/app/guides/css-in-js
https://www.joshwcomeau.com/react/css-in-rsc/

CSS-in-JS Example

https://docs.tss-react.dev/

```
import { tss } from "tss-react/mui";
import Button from "@mui/material/Button";
import { useState } from "react";

type Props = {
    className?: string;
};

export function MyButton(props: Props) {
    const { className } = props;

    const [isClicked, setIsClicked] = useState(false);

    const { classes, cx } = useStyles({ color: isClicked ? "blue": "red" });

    //Thanks to cx, className will take priority over classes.root 🤩
    return (
        <Button
            className={cx(classes.root, className)}
            onClick={()=> setIsClicked(true)}
        >
            hello world
        </Button>
    );
}

const useStyles = tss
    .withParams<{ color: "red" | "blue"; }>()
    .create(({ theme, color })=> ({
        root: {
            // The color of the text is either blue or red depending of
            // the state fo the component.
            color,
            // When the curser is over the button has a black border
            "&:hover": {
                border: '4px solid black'
            },
            // On screens bigger than MD the button will have a big cyan border
            [theme.breakpoints.up("md")]: {
                border: '10px solid cyan'
            }
        }
    }));
```

CSS-in-JS Example

https://emotion.sh/

https://styled-components.com/

```
/** @jsx jsx */
import React from 'react';
import {css, jsx} from '@emotion/core'
import styled from 'styled-components'

const Title = styled.h1`
  color: brown;
`;

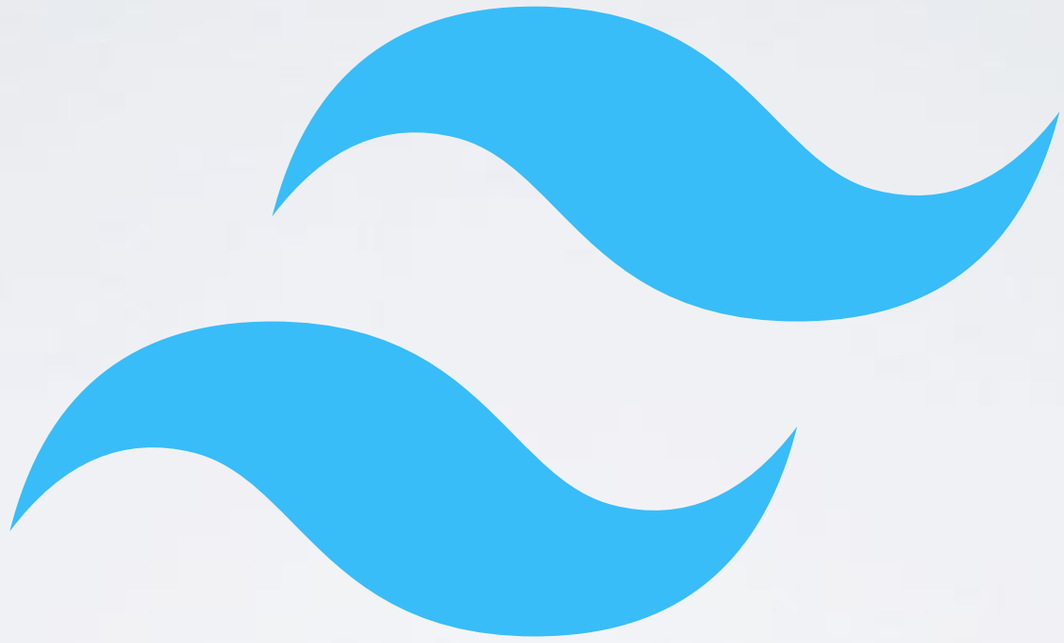
export default function Greeter() {
  return (
    <div>
     <h1 css={css`
        color: pink;
      `}>Styled with Emotion</h1>
      <Title>Styled with Styled components</Title>
    </div>
  )
}
```

emotion

styled components

Note: typically it does not make sense to use different styling libraries!

Love it or hate it:



TailwindCSS

... but it might become a defacto-standard :-)



TailwindCSS

utility-first CSS framework

```
<button class="px-4 py-1 text-sm text-purple-600 font-semibold rounded-full border
               border-purple-200 hover:text-white hover:bg-purple-600
               hover:border-transparent focus:outline-none focus:ring-2
               focus:ring-purple-600 focus:ring-offset-2">
    Message
</button>
```

https://tailwindcss.com/docs/utility-first#why-not-just-use-inline-styles

Framework agnostic but especially popular in React and other JSX-based frameworks:
Installation for any framework:
https://tailwindcss.com/docs/installation/framework-guides

Tailwind is traditionally strong for styling raw html elements. Typically it can't be used to style a traditional component library.
But the rise of headless component libraries opened a new usage-scenario for Tailwind.

https://tailwindcss.com/

Tailwind is very controversial

Tailwind CSS is the worst:
https://www.youtube.com/watch?v=IHZwlzOUOZ4

The Tailwind CSS Drama Your Users Don't Care About
https://www.builder.io/blog/the-tailwind-css-drama-your-users-don't-care-about

Why I don't like Tailwind:
https://www.aleksandrhovhannisyan.com/blog/why-i-dont-like-tailwind-css/

Why I don't recommend Tailwind CSS
https://en.andros.dev/blog/af3ee191/why-i-dont-recommend-tailwind-css/?via=dailydev



Tailwind Controversy

```css
.button {
  background-color: blue;
  color: white;
  padding: 0.5rem 1rem;
  border-radius: 0.25rem;
}
```

vs

```
<button class="bg-blue-500 text-white px-4 py-2 rounded">
```

Tailwind arguments:
faster development speed, no CSS naming debates, easier refactoring (just delete the HTML, no orphaned CSS), and built-in design constraints that keep things consistent.

Critisim:
Duplication/Not DRY
Counter Argument:
-> in a "CSS-approach", the duplication is often in CSS
-> can be addressed with components

Tailwind Controversy

CSS

```
<!-- Week 1: Two similar cards -->
<div class="card">...</div>
<div class="card">...</div>


<!-- Week 2: Requirements change -->
<div class="card">...</div>
<div class="card-no-padding">...</div>
```

VS

```
<div class="bg-white rounded shadow p-6">...</div>
<div class="bg-white rounded shadow p-6">...</div>

<!-- Week 2 - just remove p-6 -->
<div class="bg-white rounded shadow p-6">...</div>
<div class="bg-white rounded shadow">...</div>
```

probably more duplication in css

The Real Question:
The debate isn't really about duplication - it's about where complexity lives;

CSS Traditional CSS:

Complexity in CSS (class names, specificity, cascading)
Simple HTML
Hidden relationships

VS

Tailwind:

Complexity in HTML (longer class lists)
Simple CSS (just utilities)
Explicit relationships

Thank you!

Have fun in the Web Dev Ecosystem!





Jonas Bandi
JavaScript / Angular / React / Vue / Vaadin
Schulung / Beratung / Coaching / Reviews

jonas.bandi@ivorycode.com